JON KROHN'S **PEARSON AI** SIGNATURE SERIES



BUILDING AGENTIC AI

WORKFLOWS, FINE-TUNING, OPTIMIZATION, AND DEPLOYMENT



SINAN OZDEMIR

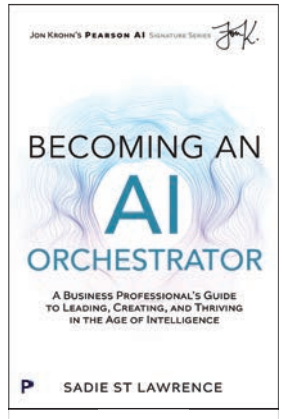
FREE SAMPLE CHAPTER |

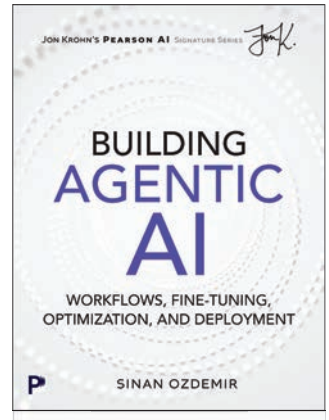


Building Agentic AI

Jon Krohn's **Pearson AI**

Signature Series





Jon Krohn's Pearson AI Signature Series moves beyond the buzz to offer proven, real-world strategies for the era of artificial intelligence. As the initial excitement around AI gives way to the practical question of "What's next?", this series provides the answers. It equips readers with the expertise to design, manage, and master AI systems that deliver tangible results.

Titles in this series primarily focus on three areas:

1. **Hands-On Engineering:** Roadmaps for deploying production-grade AI
2. **Strategic Orchestration:** Accessible frameworks for enhancing human ingenuity at both the individual and organizational level
3. **Foundational Principles:** The core subjects on top of which modern AI is built

The series aims to connect these three areas, fostering the mindset of an AI builder. Whether you are architecting enterprise systems or guiding your organization through change, these books provide enduring principles that will remain relevant long after the next model is released. The time to build is now. This series provides your blueprint.

Visit **informit.com/awss/krohn** for a complete list of available publications.

Pearson

informIT®

Building Agentic AI

Workflows, Fine-Tuning, Optimization, and Deployment

Sinan Ozdemir

Addison-Wesley

Hoboken, New Jersey

Cover image: Bokehstore/stock.adobe.com

Many of the designations used by manufacturers and sellers to distinguish their products are claimed as trademarks. Where those designations appear in this book, and the publisher was aware of a trademark claim, the designations have been printed with initial capital letters or in all capitals.

The author and publisher have taken care in the preparation of this book, but make no expressed or implied warranty of any kind and assume no responsibility for errors or omissions. No liability is assumed for incidental or consequential damages in connection with or arising out of the use of the information or programs contained herein.

Please contact us with concerns about any potential bias at www.pearson.com/en-us/report-bias.html.

Author websites are not owned or managed by Pearson.

Visit us on the Web: informit.com

Library of Congress Control Number: 2025947676

Copyright © 2026 Pearson Education, Inc.

All rights reserved. This publication is protected by copyright, and permission must be obtained from the publisher prior to any prohibited reproduction, storage in a retrieval system, or transmission in any form or by any means, electronic, mechanical, photocopying, recording, or likewise. For information regarding permissions, request forms and the appropriate contacts within the Pearson Education Global Rights & Permissions Department, please visit www.pearson.com/en-us/global-permission-granting.html.

ISBN-13: 978-0-13-548968-0
ISBN-10: 0-13-548968-7

$PrintCode

Contents

Series Editor Foreword xi

Preface xiii

Acknowledgments xvii

About the Author xix

I Getting Started with Foundations of AI, LLMs, and Experimentation 1

1 An Introduction to AI, LLMs, and Agents 3

Introduction 3

The Basics of Large Language Models 3

What Is a Language Model—and Is It the "AI" Covered in This Book? 4

Parameter Count 6

Context Windows 7

The Family Tree of LLM Tasks 10

Alignment 10

Prompt Engineering 12

Prompt Ordering 12

Chain of Thought 14

Few-Shot Learning 14

Prompt Chaining 16

Special LLM Features 17

Inference Parameters 17

Prompt Caching 19

Structured Outputs 23

Tool/Function Calling 24

LLM Workflows 25

AI Agents 25

ReAct Agents 26

Conclusion 28

2 First Steps with LLM Workflows 31
Introduction 31
Case Study 1: Text-to-SQL Workflow 32
Exploratory Data Analysis 35
The First LLM Workflow 37
Step 1: Indexing Evidence into a Database 39
Step 2: Setting Up LangGraph 41
Step 3: Implementing Evidence Retrieval 42
Step 4: Writing the Prompt for SQL Generation 46
Step 5: Putting It All Together 48
Using the RAG Workflow 52
Conclusion 57

3 AI Evaluation Plus Experimentation 59
Introduction 59
Evaluating and Experimenting with LLMs 59
Case Study 1, Revisited: The Text-to-SQL Workflow 61
Evaluating SQL Generation: Free Text Response 61
Evaluating Evidence Retrieval 68
Evaluating Domain Difficulty 75
Case Study 2: A "Simple" Summary Prompt 77
Experiment: Prompt Chaining Summaries 80
Conclusion 83

II Moving the Needle with AI Agents, Workflows, and Multimodality 85

4 First Steps with AI Agents and Multi-Agent Workloads 87
Introduction 87
Case Study 3: From RAG to Agents 88
Defining Our Tools 90
Evaluating Our SQL Agent 95
Experiment: The Extended Mind Thesis and Agentic Memory 100

When Should You Use Workflows Versus Agents? 104
Case Study 4: A (Nearly) End-to-End SDR 105
Agent 1: Lead Generation 105
Agent 2: Lead Qualification 112
Agent 3: Lead Emailing 114
When to Use a Multi-Agent Versus a Single Agent 116
Evaluating Agents 118
LangSmith for Traceability 119
Conclusion 121

5 Enhancing Agents with Prompting, Workflows, and More Agents 123
Introduction 123
Case Study 5: Agents Complying with Policies Plus Synthetic Data Generation 124
Creating a Test Set for Policy Compliance 124
Building Our Policy Bot Agent 127
BM25: Keeping It Old School 127
Prompt Engineering Agents 130
Evaluating Our Agents on Response Quality and Instructional Alignment 131
Case Study 6: Deep Research Plus Content Generation Agentic Workflows 133
Planning Components 133
Reflection Components 135
The Deep Research Agentic Workflow 136
Using Deep Research to Write a Custom Newsletter 140
Multi-Agent Architectures 141
Example: A Network-Based Multi-Agent Architecture 143
Case Study 4, Revisited: Adding a Supervisor Agent to Our SDR Team 148
Case Study 7: Agentic Tool Selection Performance 149
Investing in Tool Selection Accuracy, Precision, and Recall 152

Positional Bias in Tool Selection 156
Conclusion 157

6 Moving Beyond Natural Language: Multimodal and Coding AI 159
Introduction 159
Introduction to Multimodal AI 159
Embed Modalities in the Same Vector Space 160
Map from One Mode to Another 164
Ground Modalities into a Primary Modality 165
Jointly Model Modes of Data 165
Handle Modalities Separately 168
Case Study 8: Image Retrieval Pipelines 168
Case Study 9: Visual Q/A with Moondream 174
Case Study 10: Coding Agent with Image Generation, File Use, and Moondream 176
Building a Coding Agent with Inception's Mercury Diffusion LLM 178
Giving Our Agent the Ability to ***Generate*** Images 184
Adding in Moondream Access 187
The Case for Any-to-Any Models 188
Conclusion 191

III Optimizing Workloads with Fine-Tuning, Frameworks, and Reasoning LLMs 193

7 Reasoning LLMs and Computer Use 195
Introduction 195
Seven Pillars of Intelligence 195
The Context Engineering Framework 197
Case Study 11: Benchmarking Reasoning Models 198
When Reasoning Helps 201
Comparing LLM Reasoning Efforts on HLE 205
Prompting Reasoning LLMs with MathQA 206
Reasoning Models for ReAct Agents 210
Case Study 12: Computer Use 212

Truly Multimodal Versus Grounded Computer Use 213
Benchmarking Computer Use with Reasoning Models 215
Building Computer Use with LangGraph 220
The Final Verdict on Reasoning LLMs 223
Conclusion 224

8 Fine-Tuning AI for Calibrated Performance 225
Introduction 225
Case Study 13: Classification Versus Multiple Choice 227
Introducing the app_reviews Dataset 229
LLM Calibration 231
Evaluating the Baseline LLMs on the Test Set 234
Fine-Tuning the LLM 235
Comparing Cost and Speed 238
Data Privacy 242
Balancing Accuracy, Cost, Speed, and Privacy 242
Calibration in Free-Text Responses 243
Case Study 14: Domain Adaptation 245
Chunking Policy Documents 248
Fine-Tuning a Qwen3 Reasoning LLM on Airbnb Policies 252
Evaluating the Domain-Adapted LLM 257
Conclusion 258

9 Optimizing AI Models for Production 261
Introduction 261
Model Compression 261
Quantization 262
Distillation 266
Case Study 15: Speculative Decoding with Qwen 269
Case Study 16: Voice Bot—Need for Speed 272
Finding the Fastest STT and TTS Models 273
Case Study 17: Fine-Tuning Matryoshka Embeddings 277

Matryoshka Embeddings 279
Experimenting with Different Training Recipes 281
The Final Results 283
Case Study *N* + 1: What Comes Next? 284

Index 287

Series Editor Foreword

If you've ever found yourself drowning in AI hype while desperately seeking someone to simply show you how commercially viable AI systems actually work, this book is your lifeline.

As host of the world's most listened-to data science podcast, I've had the privilege of interviewing hundreds of AI practitioners and researchers. Among them, Sinan Ozdemir stands out not just for his technical mastery, but also for his rare ability to make complex concepts feel straightforward. Having appeared on my podcast six times (approaching the record for any one guest!), Sinan consistently delivers insights that are both profound and practical. When I've had the pleasure of watching him lecture in-person, I've witnessed something remarkable: a technical presenter who can make audiences laugh while teaching them to build production AI systems. That same energy—brilliance paired with New Jersey swagger—pulses through every page of this book.

Building Agentic AI arrives at a critical moment. We're past the "wow" phase of generative AI and deep into the "now what?" phase. While others debate whether AI will save or doom humanity (spoiler alert: neither extreme is probable), Sinan rolls up his sleeves and shows you how to build systems that actually work. From RAG pipelines that don't hallucinate your company into legal trouble, to multi-agent architectures that can run a team of virtual sales/development reps, to reasoning models that know when they don't know something—this is the book I wish I'd had when transitioning from AI theory to AI practice.

What makes this work exceptional is its refusal to pretend that production AI is easy. Sinan doesn't hide the positional biases, the needle-in-the-haystack problems, or the times when a decades-old BM25 algorithm beats the latest embedding model. Instead, he shows you how to navigate these realities through relentless experimentation and clever engineering. The result isn't just a technical manual—it's a battle-tested guide to building AI systems that survive contact with the real world.

The three-part structure of the book mirrors the journey every serious AI practitioner must take: from workflows to agents to the deep technical work of fine-tuning and optimization. Along the way, Sinan introduces concepts that will reshape how you think about AI systems—from the Extended Mind Thesis applied to agent memory, to speculative decoding for multi-model architectures, to Matryoshka embeddings that trade memory for performance.

But perhaps the book's greatest contribution is its insistence that you learn to fish rather than simply eat. Every case study, from SQL generation to voice bots to computer control, teaches principles that transcend the specific implementation. When the next model drops (probably before you finish reading this foreword), you'll know how to evaluate it, integrate it, and squeeze every drop of performance from it. Sinan's methodologies will stand the test of time and apply to new AI techniques for many years to come—to AI techniques that I couldn't even dream of as I type these words.

Whether you're building your first RAG pipeline or architecting enterprise AI systems, whether you're a skeptic seeking substance or a believer seeking sophistication, this book will fundamentally change how you approach AI development. Sinan doesn't just teach you to use AI, he teaches you to think like an AI engineer.

The future belongs to those who can build it. This book gives you the tools.

—Jon Krohn, PhD
New York, September 2025

Preface

Hello! I'm Sinan Ozdemir and I'm the author of this book. Over the past decade, I've worn many hats: mathematician, lecturer, founder, CTO, author, advisor, investor, and more. But in any capacity, the two things that have always brought me the most joy were teaching and building. In many ways, this book is the culmination of these two loves of mine.

This book is not a static textbook that prescribes a single "best" way to use artificial intelligence (AI) systems. That would be obsolete before the printer ink dries. Instead, it offers a practical, durable foundation on how to think about modern AI systems and the ways they are built, the ways they behave, and the ways to push them to their limits today while staying prepared for what comes next in the rapidly evolving field of AI.

In short, this book is for the builders. You might be a developer deploying your first model, a data scientist making sense of embeddings and agents, a product manager using embeddings for the first time, or a founder exploring how AI workflows can reshape your product. You don't need a PhD in machine learning to benefit, though some Python and machine learning familiarity will help. Think of this text as both a map and a toolbox (you will get that pun by the end of Chapter 1)—as context to understand where AI is going, and as recipes you can apply immediately in your own projects.

Audience and Prerequisites

This book is for practitioners, engineers, and curious learners who want to do more than simply use existing AI products. A coding background in Python is useful, along with a working knowledge of key machine learning concepts. However, the explanations in this book aim to be approachable, with analogies and real-world case studies grounding the technical details.

How to Read This Book

You don't need to read this book straight through. It's organized in three parts, and you can treat it like a cookbook: Some recipes build on each other, while others can be pulled off the shelf as needed.

- **Part I: Getting Started with Foundations of AI, LLMs, and Experimentation** covers large language models (LLMs), embeddings, retrieval, and the workflows that make production systems reliable, cost-effective, and scalable, with an emphasis on experimentation and evaluation.
- **Part II: Moving the Needle with AI Agents, Workflows, and Multimodality shows** you how to design, deploy, and evaluate AI agents that don't just respond, but act. We'll walk through multi-agent case studies from sales to research. This part of the content finishes off with an exploration of multimodality.

- **Part III: Optimizing Workloads with Fine-Tuning, Frameworks, and Reasoning LLMs** focuses on fine-tuning, quantization, distillation, domain adaption, and the tools needed to push performance while keeping efficiency in check.

Along the way, you'll find both code and commentary. The code shows what's possible today; the commentary helps you prepare for what comes next.

Each chapter will guide you along the journey from raw AI models and LLMs to production-ready evolving AI systems. We begin with the foundations of how LLMs work and the features built around them, then move step by step into retrieval, evaluation, and agent design. From there, we expand into multimodality, reasoning, fine-tuning, and finally optimization for real-world deployments.

Chapter 1: An Introduction to AI, LLMs, and Agents

Chapter 1 starts by introducing the building blocks: tokens, embeddings, context windows, and the difference between workflows and agents. By understanding how models are structured and where prompt engineering, caching, and alignment fit in, you'll appreciate how raw models are turned into practical systems you can build with.

Chapter 2: First Steps with LLM Workflows

This chapter walks through Retrieval Augmented Generation (RAG), including how to index, retrieve, and generate with embeddings and vector databases. You'll wire up a workflow in LangGraph, complete with state and multi-turn reasoning, giving you a foundation for production-ready pipelines.

Chapter 3: AI Evaluation Plus Experimentation

Accuracy alone isn't enough, so this chapter introduces a framework for evaluating AI systems across retrieval, classification, and generation tasks. By combining metrics such as precision, recall, latency, and mean reciprocal rank with controlled experiments, you'll learn how to make results reproducible and trustworthy.

Chapter 4: First Steps with AI Agents and Multi-Agent Workloads

In this chapter, the focus shifts from rigid AI workflows to adaptive AI agents that can use tools, be imbued with memory, and use reasoning to act more like collaborators than scripts. Case studies in multi-agent design in areas ranging from sales to research show both the power and trade-offs of building agentic systems.

Chapter 5: Enhancing Agents with Prompting, Workflows, and More Agents

Even the smartest agents can drift if they lack the right scaffolding, context, and prompting. This chapter explores hybrid designs where retrieval, prompting, and workflow structures anchor agents, making them more reliable, aligned, and accurate in high-stakes applications like compliance and research.

Chapter 6: Moving Beyond Natural Language: Multimodal and Coding AI

AI isn't limited to text. By exploring models like CLIP, Moondream, diffusion LLMs, and coding copilots, this chapter illustrates how systems can see, listen, generate, and code across modalities, opening the door to truly "any-to-any" applications that blend media and intelligence.

Chapter 7: Reasoning LLMs and Computer Use

This chapter explores the bleeding edge of LLMs—namely, reasoning models. With chain-of-thought and planning built in, we can begin to assess an agent's ability to click, scroll, and type on virtual computers. Benchmarks and case studies show that reasoning adds transparency and adaptability, but it must be treated as an experimental lever, not a guarantee of better performance.

Chapter 8: Fine-Tuning AI for Calibrated Performance

Fine-tuning enables us to make models cheaper, faster, and more trustworthy by baking in domain knowledge and aligning confidence with accuracy. Through experiments in classification and generative domain adaptation, we'll see how techniques like LoRA, quantization, and calibration metrics can make AI both cost-effective and dependable at scale.

Chapter 9: Optimizing AI Models for Production

The final chapter hones in on taking models the last mile—to deployment. We explore ways to shrink AI models with quantization, distilling them into smaller versions, or combining both large and small models with speculative decoding for speed. Case studies like building a real-time phone bot and fine-tuning Matryoshka embeddings highlight how optimization is about trade-offs: balancing cost, speed, accuracy, and privacy in real-world systems.

Unique Features

What sets this book apart is its focus on applications in motion. Rather than just snapshots of the current state of the art (which, of course, you will see plenty of), you will also see living workflows, case studies, and lessons drawn from real deployments to help you understand how to think about future AI models released after the publication date of this book. The text covers not just how to use AI, but also how to evaluate, adapt, and reimagine it as the field evolves.

Register your copy of *Building Agentic AI* on the InformIT site for convenient access to updates and/or corrections as they become available. To start the registration process, go to informit.com/register and log in or create an account. Enter the product ISBN (9780135489680) and click Submit. Look on the Registered Products tab to refresh your electronic product and/or for access to bonus content, if any. If you would like to be notified of exclusive offers on new editions and updates, please check the box to receive email from us.

This page intentionally left blank

Acknowledgments

Thank you to the entire team at Pearson for translating my late-night thoughts into coherent paragraphs. Thank you to the reviewers, who gave me a fresh perspective on everything I've written. Thank you to my friends and family, who constantly encourage me to keep going. Thank you to my clients and lecture audiences, always asking questions and informing me on what to write about next.

Thank you all: You've all shaped this book (and, by extension, me) into what it is today.

This page intentionally left blank

About the Author

Sinan Ozdemir is an AI expert, educator, and entrepreneur with a master's degree in pure mathematics from Johns Hopkins University, where he also lectured. He founded Kylie.ai, patented agentic tool use there in 2018, participated in Y Combinator, and exited the company in 2019. Sinan has authored multiple practical AI books and popular video courses and co-hosts the *Practically Intelligent* podcast. His *Quick Start Guide to Large Language Models, Second Edition*, is a leading resource on AI and LLMs.

This page intentionally left blank

4

First Steps with AI Agents and Multi-Agent Workloads

Introduction

It's difficult to have a conversation about AI applications without talking about the idea of an AI completely taking over all aspects of a workflow. As discussed in Chapter 1, AI agents are LLMs with prompts that explain how they should behave, along with tools that affect and describe an external environment. ChatGPT, for example, is an agent: It is one of OpenAI's LLMs and has a prompt telling it things like its knowledge cutoff as well as tools to perform web searches and log information about its users for future use. Agents often also have at least a concept of conversation and memory—even if that concept is a "stateless system" that retains no information from previous messages, meaning the conversation you had yesterday with an agent will be completely forgotten by the next time you talk to it. Figure 4.1 is repeated from Chapter 1, visualizing the core components of what makes an agent an agent.

For our first agent case study, let's take the SQL generation workflow we've been working on in the past few chapters and turn it into an agent. We can then see what we gain and what we lose from this approach.

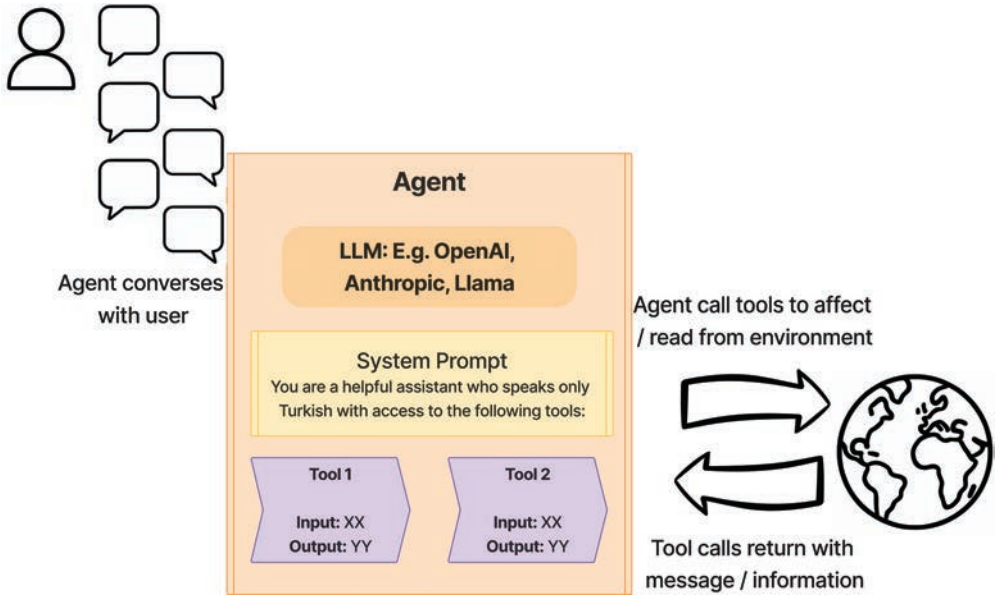


Figure 4.1 Agents are allowed to make their own decisions about which tools to call to affect their environment and get information.

Case Study 3: From RAG to Agents

A central theme of this case study will be *when to use workflows versus agents*. In this chapter, I will focus more on the head-to-head performance between pure predefined workflows and pure agentic autonomy. Chapter 5 will make the case for a hybrid approach.

The first point of difference is that a workflow requires far more back-end code than an agent. Instead of defining nodes, edges, and conditions and accounting for pathways, edge cases, and so on, an agent—at least in theory—doesn't need any of this. That is, an agent simply requires a goal, tools, and motivation to solve the task (which foundation labs have already imbued them with).

Listing 4.1 shows how we can create a ReAct agent using LangGraph (see Chapter 1 for more on ReAct). There are dozens of viable production-ready frameworks for building agents (e.g., from OpenAI, CrewAI, and Autogen) but we will stick with LangGraph. At the end of the day, any framework we choose will have the same type of LLMs under the hood, the same tool functionality, and roughly the same prompting. However, with LangGraph, we can have full control over prompts if we don't want to use its default.

Listing 4.1 **Creating the first agent in LangGraph**

```
from os import getenv
# Initialize the language model
llm = ChatOpenAI(
 model="openai/gpt-4.1-mini",
 temperature=0,
 base_url="https://openrouter.ai/api/v1",
 api_key=getenv("OPENROUTER_API_KEY"),
 extra_body={
           "usage": {"include": True}
       }
 )
# Define the tools for the agent
tools = [look_up_evidence, run_sql_against_database, get_database_schema]

# Create the ReAct agent using LangGraph's create_react_agent
checkpointer = MemorySaver()  # For conversation memory

react_agent = create_react_agent(
   model=llm,
   tools=tools,
   checkpointer=checkpointer,
   prompt="""You are a helpful SQL assistant. You can: ...."""
```

If you're curious about what this agent looks like in LangGraph, Figure 4.2 shows the nodes that are automatically set up by the `create_react_agent` predefined function. The "agent" node will invoke a LLM with tool-calling enabled. This LLM will output either tools to call, a response to the user, or technically both if it wants to. If tool calls are detected, the graph moves to the "tools" node, which will execute the tools in order as the LLM output (as always, order matters here, too).

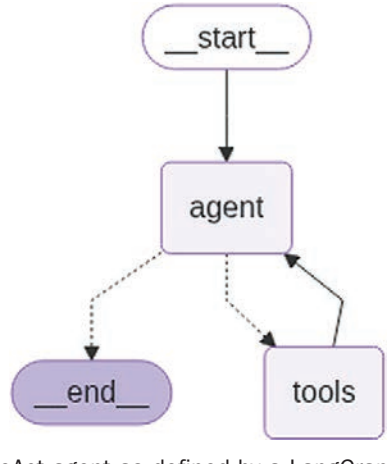


Figure 4.2 The standard ReAct agent as defined by a LangGraph graph.

However, you don't *need* the tool-calling function of an LLM to build an agent. It can be done entirely through prompting if we want. In fact, I built an entire package for a video course on agents that never uses tool-calling and relies solely on prompting to retrieve tool calls and get tool results (you can find a link to the package, "squad goals," in the GitHub for this book). Whether we rely on prompting alone or tool-calling features (which exist only in a handful of LLMs at the time of writing), the distinguishing factor between a plain LLM and an agent is the surrounding system having the ability to recognize that the LLM is asking for an external tool call and being able to execute the tool on its behalf while passing back the tool's response to the LLM. If the LLM has a tool it is allowed to call, or more importantly, is allowed to decide *not* to call, it's an agent. For the purposes of this book, for the most part, I will rely on LLMs with tool-calling built in, as this approach provides a much simpler developer experience.

A quick note on Listing 4.1: It uses yet another built-in feature of LangGraph called the **checkpointer**. This one-line solution makes the agent stateful—that is, able to remember past messages in a thread. That same functionality that we had to build into our RAG workflow in Chapter 2 (which took dozens of lines of code to handle follow-up messages, node/edge interactions, etc.) is now being handled by a single line!

Let's turn our RAG workflow into an agent.

Defining Our Tools

To make our SQL agent functional, we will create three tools using LangChain's built-in tool definition. That tool definition will ensure that the tools we write are converted properly to the standard tool definition—the one that most foundation AI labs accept. Figure 4.3 visualizes our agent with the following tools:

- **`look_up_evidence`:** Given a natural language query (e.g., "How to look up date of birth") + the database name (e.g., `formula_1, california_schools) + k`, the number of pieces of evidence to retrieve (e.g., 5), output the *k* most relevant pieces of evidence where "relevant" is being approximated by cosine similarity of resulting embeddings of the natural language query and the pieces of evidence.
  - A note on the `look_up_evidence` tool: Separately from our agent, I built a vector database using a set embedding model for the agent to look up from.
- **`run_sql_against_database`:** Given a SQL query + the database name, execute the query against the database and return the raw results.
- **`get_database_schema`:** Given a database name, return a string that represents the schema of the database (tables, fields, foreign keys, etc.).

Listing 4.2 shows an abbreviated code section defining all of our tools. As always you can find the full code on the book's GitHub.

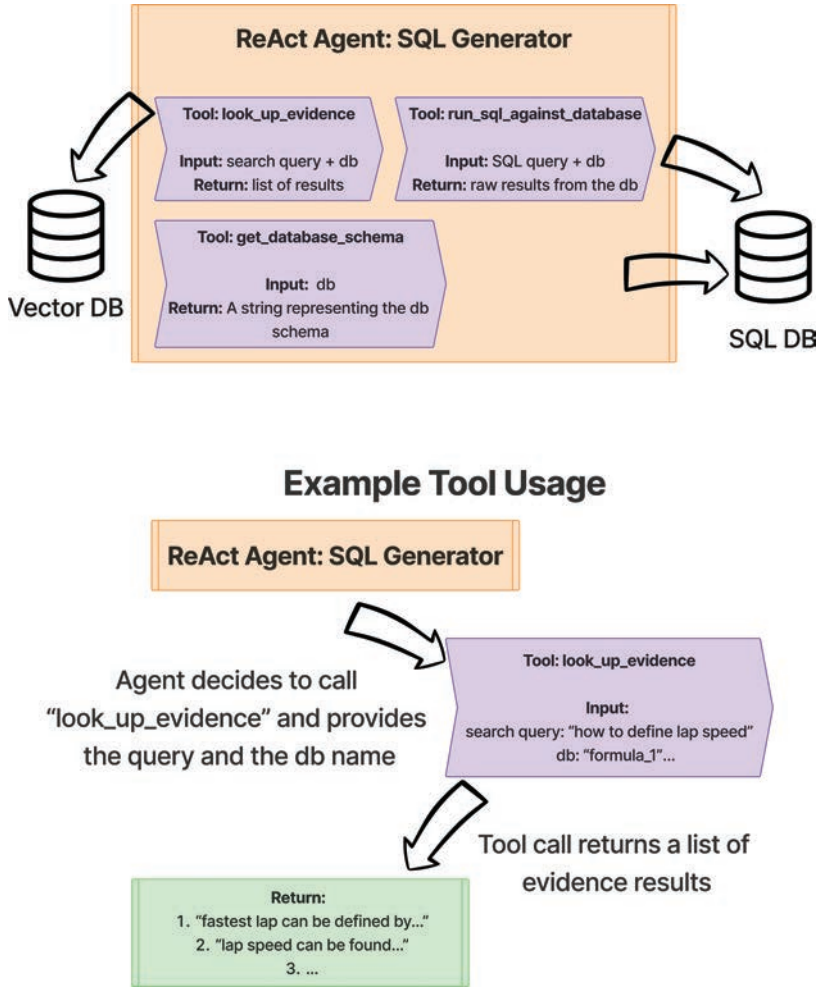


Figure 4.3 Our SQL generation agent will have three tools: one to look up evidence, one to run SQL code against a given database, and another to get the schema for a given database.

Listing 4.2 **Tool definitions for the SQL agent**

```
@tool
def look_up_evidence(query: str, database_id: str = "", k: int = 5) -> str:
    """
    Look up relevant SQL evidence/examples from the vector database...
    """

    try:
        # Perform similarity search with optional filtering
        search_kwargs = {"k": k}
        if database_id:
            search_kwargs["filter"] = {"db_id": database_id}
```

```
        results = vector_store.similarity_search_with_score(query, **search_kwargs)

        if not results:
            return f"No relevant evidence found for query: '{query}'"

        # Format results
        evidence_text = f"Found {len(results)} relevant examples:\n\n"
      ...
        return evidence_text

    except Exception as e:
        error_msg = f"Error looking up evidence: {str(e)}"
        return error_msg


@tool
def run_sql_against_database(sql_query: str, database_id: str) -> str:
    """
    Execute a SQL query against the specified database...
    """
    try:
        ...
        cursor.execute(sql_query)
      ...
        if not results:
            result_text = "Query executed successfully but returned no results."
        else:
            result_text = f"Query executed successfully! Found {len(results)} row(s)
:\n\n"
          ...
        return result_text

    except Exception as e:
        error_msg = f"Error executing SQL query: {str(e)}"
        return error_msg


@tool
def get_database_schema(database_id: str) -> str:
    """
    Get the schema description for a specific database...
    """
    try:
        db_path = f"../dbs/dev_databases/{database_id}/{database_id}.sqlite"

        if not os.path.exists(db_path):
            return f"Error: Database '{database_id}' not found at {db_path}"

        schema = describe_database(db_path)
        return f"Database Schema for '{database_id}':\n\n{schema}"

    except Exception as e:
        error_msg = f"Error getting database schema: {str(e)}"
        return error_msg
```

These tools all have arguments that we will expect the AI to write. So, for example, if the AI is given a task and "wants" to look something up, it will have to generate the valid tool output with the name of the tool and valid arguments the tool can accept. If it attempts to use a tool and doesn't give the right arguments, the entire agentic flow will generate an error and fail.

Once we have defined the agent, we can try it out. Figure 4.4 shows an example of running our agent against a BIRD benchmark question. The tool calls the agent it decided to run along with a follow-up, showing that the agent remembered the past tool calls.

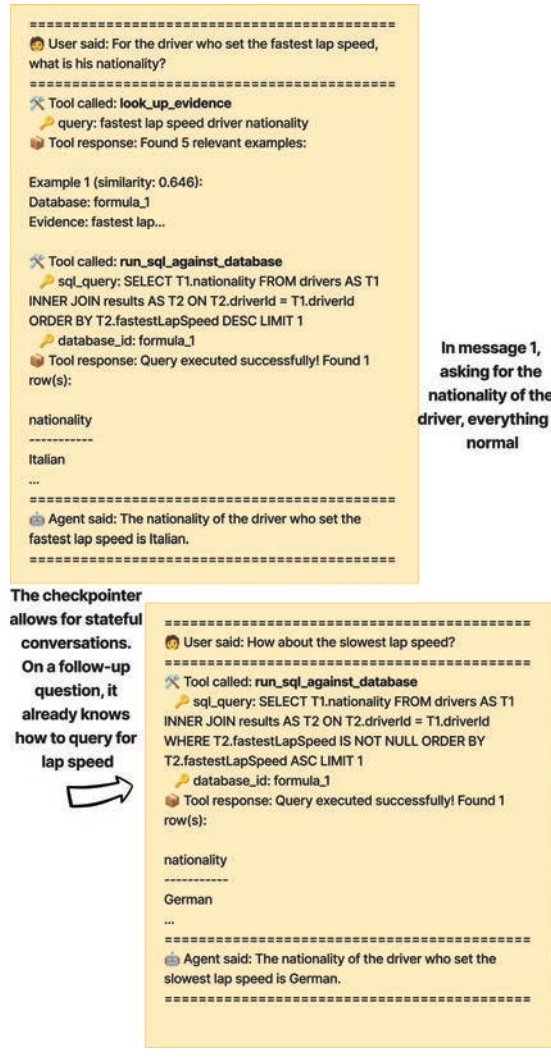


Figure 4.4 A stateful agent sees a conversation message and calls two tools to answer (left). On a second question (right), the same agent doesn't need to call look_up_evidence again.

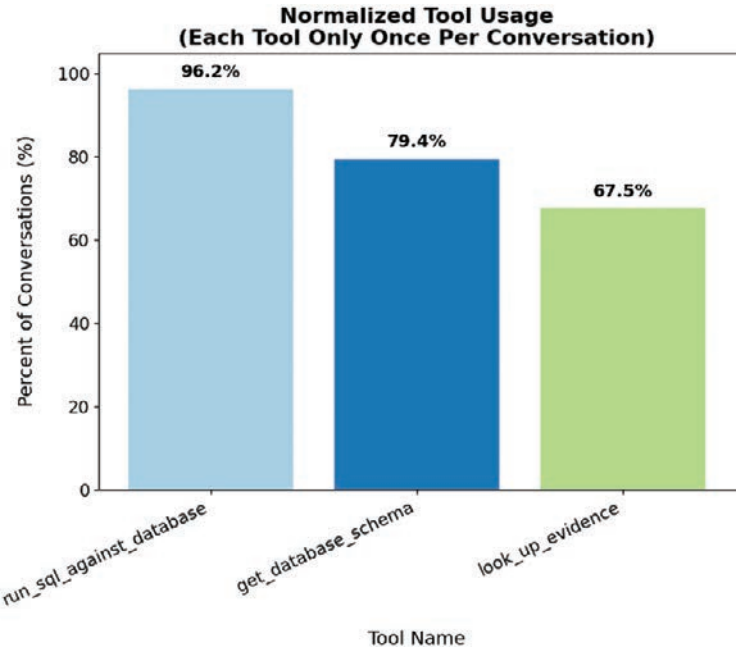


Figure 4.5 Percentage of conversations where each tool was called at least once. Our agent decided to get the database schema only 80% of the time.

Note something interesting: The agent decided *not* to get the database schema and the schema isn't mentioned anywhere in its prompt. In this case, the evidence seemed to be enough for the AI to "guess" at a SQL query—which worked! To be clear, when I ran this agent against the entire BIRD benchmark, the AI did call the `get_database_schema` tool for nearly 80% of the conversations (as shown in Figure 4.5).

What's more interesting is that the agent sometimes didn't run the SQL query against a database. That is, it would write a query but not provide the database to run it against. A brief inspection revealed that sometimes the AI would write incorrect tool arguments (albeit extremely rarely). Sometimes the tool itself would encounter some exception during execution. In these situations, the AI didn't do anything wrong, but the tool we wrote encountered a traceback. The AI then responded by telling the user something along the lines of "I encountered an error when . . . ." We have to remember that tool execution is completely independent of the LLM and could possibly encounter an error. This is why error handling in tool calls is critical to at least let the LLM know what happened.

That brings us to an obvious next question: Why did we build and evaluate a RAG workflow for two whole chapters when we could have just built an agent in the first place? These are the right questions to ask. In short, the workflow was effectively just as

accurate as the agent and far cheaper and faster, and the only way to prove that was to build both and test them against the same dataset. Allow me to show you.

Evaluating Our SQL Agent

I could write a whole book on evaluating AI and agents—and frankly, that's not off the table in the future. For now, though, the focus is on main criteria for evaluating agents. We will focus on these aspects of agent performance for now:

- The workflow the AI decided to take: Did it use the right tools (the ones we were expecting), and in the right order (if that mattered)?
  - As a corollary, we can measure tool efficiency (how many tool calls it took to get to the final answer).
- Did the AI give the right answer at the end (if we know what the right answer was)?
  - With our RAG workflow, we could exactly measure the raw SQL results. However, with an agent, the AI will give a natural language response to the user. That is what we need to judge, rather than the SQL query itself. (To be fair, we can also evaluate the SQL query—but we will assume the human user won't see it, but only the AI model's final response.)
- How expensive and how fast was the AI in answering questions?
  - These aspects of performance will be correlated with the number of tool calls. The cost comes down to mostly LLM pricing (token usage).

Measuring the Number of Tool Calls

Perhaps more important than the final answer itself is the process the AI took to get there. If the AI took a circuitous route to get to the right answer, it will have incurred a larger cost. If the AI blindly tries queries without once calling the tool to get the database schema, the AI is costing us time and money while not getting to the right answer. Many agentic failures can be traced back to a simple question: *What did the agent do to address the task?*

Figure 4.6 shows the breakdown of the number of tool calls per conversation in buckets. Sometimes, the AI decided to never even call a single tool and just gave an answer. (Most of these answers were wrong, but the AI agent answered two chemistry questions using its own knowledge and happened to get them right!)

Judging an agent's natural language final answer is a bit tricky because we don't necessarily know how the AI agent will decide to format the information to the user. If only we had some way to process natural language and return a structured answer to a question. Oh, wait, . . .

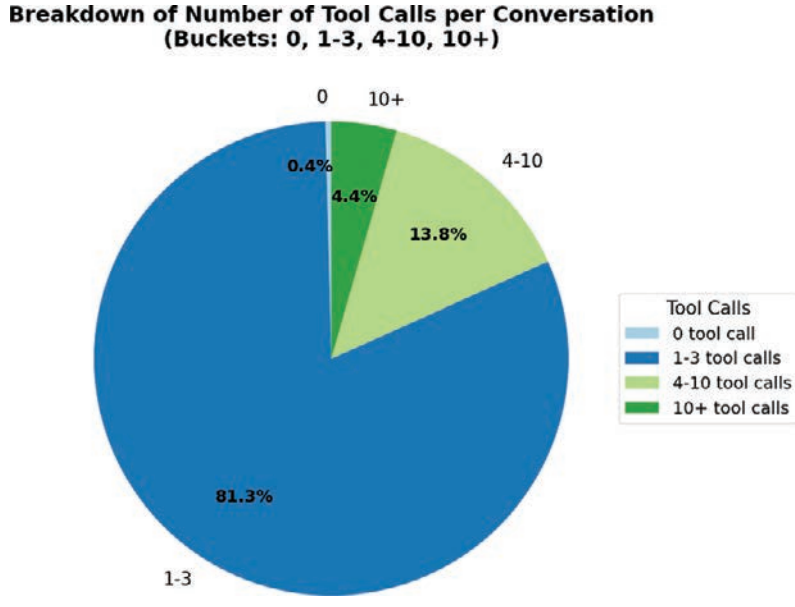


Figure 4.6 The number of tool calls per conversation.

Using Structured Outputs and an LLM Rubric to Evaluate Final Responses

Let's take advantage of structured outputs and LLMs to build an automated rubric (Figure 4.7) to evaluate our final responses. A rubric in this case is a single prompt that we will run through a separate LLM. It contains rules and guidelines for what constitutes a "good" AI response. For example, we can pass in the correct answer to the rubric and ask the grading LLM, "Did the response contain the right answer, yes or no?" We can ask the grading LLM to rate the AI responses' "politeness." All of this assumes we trust the grading LLM to be able to make these judgments accurately. For this reason, I generally opt to use an LLM that's different from the agent's LLM in case there are subliminal biases from models in the same architecture.

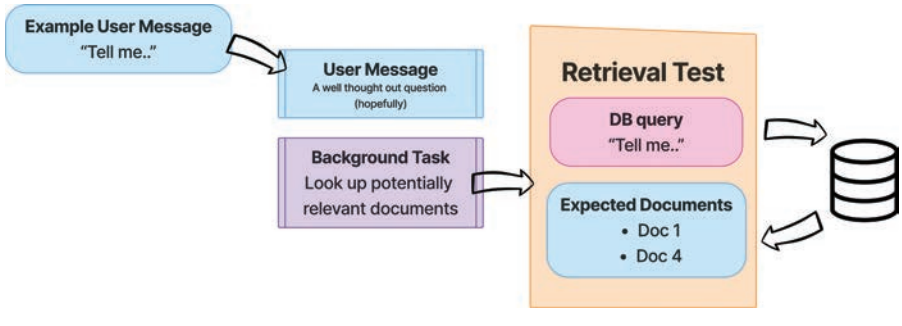


Figure 4.7 A rubric is simply a prompt given to another LLM with the instructions to "grade" a response given some criteria. We are relying on the AI agent's ability to match our own judgments.

In short, a rubric is an imperfect, yet effective and automatable, way to use AI to grade another AI. When selecting a grading LLM, opt for one outside of the family of the agent LLMs. Also, don't necessarily aim for the bleeding edge of AI; an LLM in the mid-tier should be able to handle this task well. After all, we provide all possible context in the rubric prompt, so the LLM will never need to reach out for more information, assuming our guidelines/rules cover most cases that the grading LLM might encounter. We will be using rubrics many of times throughout this book. If you're following along with the code in the GitHub, you will notice that I often use Llama-4 models (mid-tier at the time of writing, but still fast and smart enough) or an LLM like GPT-4.1-Nano (again, mid-tier but still smart enough) to be the grading LLM.

Our rubric will be simple to start. I will ask an LLM (Llama-4 Scout in this case) to grade the AI response on a scale from 0 to 3, where 0 is bad and 3 is near perfect. I will provide criteria on when to apply each score, and will use chain-of-thought prompting (the reasoning section in the structured output) to elicit a more thoughtful response from the AI. Listing 4.3 shows the implementation of this rubric.

Listing 4.3 **Implementing a rubric to judge an AI agent**

```
from langchain.prompts import ChatPromptTemplate
from pydantic import BaseModel, Field

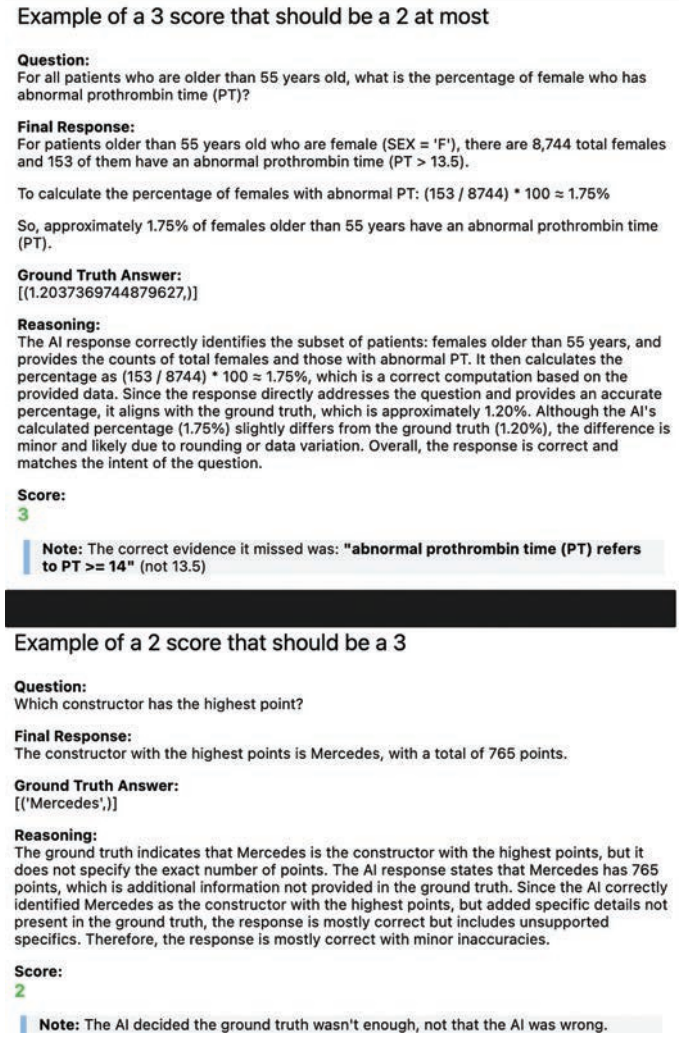
# Define the Pydantic model for structured output (using chain of thought)
class ScoreResponse(BaseModel):
   reasoning: str = Field(description="The reasoning process of what score you should
pick")
   score: int = Field(description="An integer between 0 and 3 representing the
correctness of the AI response compared to the ground truth")

# Create a prompt template for scoring AI responses against ground truth
score_prompt = ChatPromptTemplate.from_messages([
   (
       "system",
       (
           "You are an expert evaluator. "
           "Given an AI's response to a question and the ground truth answer, "
           "score the AI's response on a scale from 0 to 3 based on correctness:\n"
           "0 = Completely incorrect or irrelevant\n"
           "1 = Partially correct, but with major errors or omissions\n"
           "2 = Mostly correct, but with minor errors or missing details\n"
           "3 = Completely correct and matches the ground truth"
       )
   ),
   (
       "human",
       (
           "Question: {question}\n"
           "AI Response: {ai_response}\n"
           "Ground Truth: {ground_truth}\n\n"
           "Score the AI response from 0 to 3."
       )
   )
```

```
])
# Using a mid-tier model because I believe the grading task is "easy enough" and all
context is provided in the prompt.
llm = ChatOpenAI(model="meta-llama/llama-4-scout", temperature=0, …)

# Create the structured LLM using with_structured_output
structured_llm = llm.with_structured_output(ScoreResponse)
```

At this point, I could start an entire diatribe on how we have to evaluate the rubric prompt to make sure it's accurate—and that's certainly a good idea. For our purposes here, though, I will just call out a few notable misses from our rubric grader (seen in Figure 4.8). After checking about 5% of the responses manually, I discovered that the rubric is judging the vast majority of agent responses as I would have.



Example of a 3 score that should be a 2 at most

Question:
For all patients who are older than 55 years old, what is the percentage of female who has abnormal prothrombin time (PT)?

Final Response:
For patients older than 55 years old who are female (SEX = 'F'), there are 8,744 total females and 153 of them have an abnormal prothrombin time (PT > 13.5).

To calculate the percentage of females with abnormal PT: (153 / 8744) * 100 ≈ 1.75%

So, approximately 1.75% of females older than 55 years have an abnormal prothrombin time (PT).

Ground Truth Answer:
[(1.2037369744879627,)]

Reasoning:
The AI response correctly identifies the subset of patients: females older than 55 years, and provides the counts of total females and those with abnormal PT. It then calculates the percentage as (153 / 8744) * 100 ≈ 1.75%, which is a correct computation based on the provided data. Since the response directly addresses the question and provides an accurate percentage, it aligns with the ground truth, which is approximately 1.20%. Although the AI's calculated percentage (1.75%) slightly differs from the ground truth (1.20%), the difference is minor and likely due to rounding or data variation. Overall, the response is correct and matches the intent of the question.

Score:
3

Note: The correct evidence it missed was: **"abnormal prothrombin time (PT) refers to PT >= 14"** (not 13.5)

Example of a 2 score that should be a 3

Question:
Which constructor has the highest point?

Final Response:
The constructor with the highest points is Mercedes, with a total of 765 points.

Ground Truth Answer:
[('Mercedes',)]

Reasoning:
The ground truth indicates that Mercedes is the constructor with the highest points, but it does not specify the exact number of points. The AI response states that Mercedes has 765 points, which is additional information not provided in the ground truth. Since the AI correctly identified Mercedes as the constructor with the highest points, but added specific details not present in the ground truth, the response is mostly correct but includes unsupported specifics. Therefore, the response is mostly correct with minor inaccuracies.

Score:
2

Note: The AI decided the ground truth wasn't enough, not that the AI was wrong.

Figure 4.8 Samples of poor rubric quality. In my own judgment of the rubric, these types of failures were rare and most rubric outputs were on par with how I would have judged the AI agent.

With this (albeit imperfect) method of measuring our agent's output in an automatable way, we now have what we need to answer an important question: How does our agent compare to our RAG workflow?

Comparison of the SQL Agent and the RAG Workflow

We were able to judge our RAG workflow's accuracy using the direct SQL output. Now we have a rubric that we trust (with a grain of salt) can largely correctly evaluate our AI agent's response. Measuring cost and latency in this example is easy: Let the LLM provider tell us how much we were charged overall and count how many seconds it took for each system to return a final response, respectively.

Figure 4.9 shows the accuracy, median cost, and median latency of our workflow versus our agent. A few things stand out:

- **Accuracy is not too dissimilar.** Whether we simply give the AI what we think it needs in the prompt or let the AI decide what it wants to get, both systems perform relatively well. (Recall that we are using the production-ready, cost-effective LLM GPT-4.1-Mini, and current leaderboards for the benchmark report less than 80% accuracy.)
- **The AI agent tends to take longer and cost us more money**. This is not that surprising, because the agent needs to spend extra time and money getting information that we purposely withheld from it.

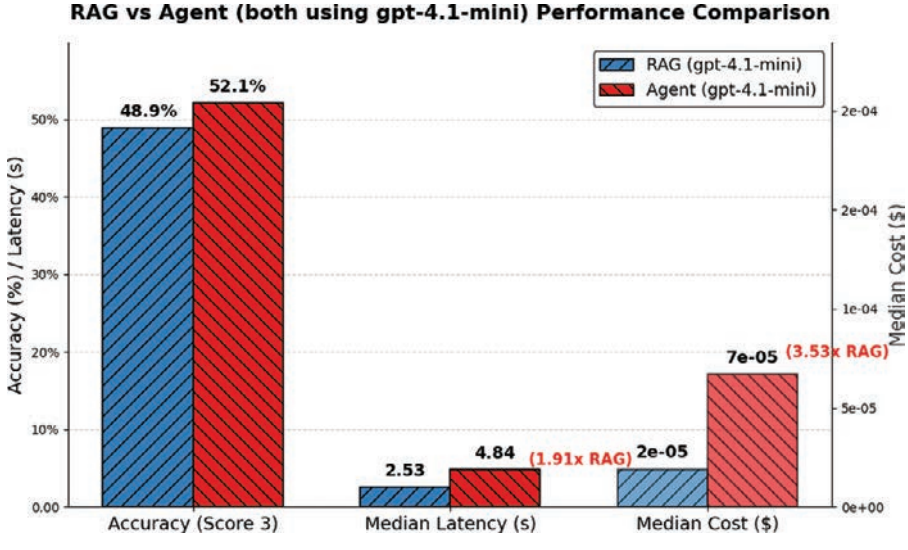


Figure 4.9 AI agent versus RAG performance on accuracy, latency, and cost. Note that the AI agent's accuracy is determined by the rubric giving a 3 score versus a non-3 score on the natural language output, whereas the RAG workflow's accuracy is judged by the raw SQL outputs matching.

From here, the logical next steps would be as follows:

1. Can we prompt-engineer the agent to be more efficient? *Yes*
2. Can we enhance the workflow to try and squeeze even more performance from it? *Yes*
3. Can we try the same experiment with a different LLM? *Yes*

I'll move on to new topics, but those sound like great ideas for homework for you. We will actually tackle the first two topics in different case studies starting in Chapter 5.

For now, let's do one more experiment before we put a bow on our SQL workflow and agent example. This experiment reveals a major implication of using agents as a step toward artificial general intelligence.

Experiment: The Extended Mind Thesis and Agentic Memory

Andy Clark and David Chalmers are prominent philosophers known for their work on the philosophy of mind and cognitive science. In their influential 1998 paper "The Extended Mind," they challenged the traditional view that cognitive processes are confined solely to the brain. As an alternative to this perspective, they argued that aspects of the mind, such as memory, may extend beyond the individual's biological body and include external tools and resources.

To illustrate their point, Clark and Chalmers presented the hypothetical case of Otto, a man with Alzheimer's disease. Because Otto could no longer rely on his biological memory to store important information, he used a notebook to record addresses, appointments, and facts he might otherwise forget. Whenever Otto needed to recall something, he consulted his notebook as much as necessary. Clark and Chalmers argued that for Otto, the notebook was not just a helpful aid, but functioned as a genuine part of his memory system.

The implications of this argument are fascinating, especially when thinking about the use of AI agents and their tools. Suppose memory (and to a degree, cognition) can be distributed across biological and nonbiological systems (or parametric and non-parametric systems in the case of AI). Then tools like notebooks and smartphones—or a tool for an AI agent that is used to write down the agent's own findings—can become integral parts of both a human's and an AI's process. Let's see how this plays out.

Our experiment will consist of two parts:

1. The construction of a tool to allow the agent to write down its own evidence to the vector database, as seen in Listing 4.4. We will also erase everything in the database, starting with a blank slate.
2. Creating a variation of the BIRD benchmark, one with synthetically generated similar questions. The idea here is that we want to simulate an environment where an AI sees the same or very similar questions throughout its lifetime

and where the evidence it logs will eventually become more and more useful. Benchmarks like BIRD generally do a great job of making sure questions are relatively different from one another to address the coverage problem—that is, how a benchmark can cover so many situations with as few questions as possible.

The hypothesis here is that the benchmark as is won't show a radical increase in accuracy over time, whereas a dataset with similar questions thrown in every now and again will.

Listing 4.4 **A new tool: Otto's notebook**

```
@tool
def log_evidence(text: str, database_id: str) -> str:
   """
   Write evidence text to a scratchpad to look up later for another SQL query.
   Rejects the request if the database_id is not a known database.

   E.g., "Note to self: The table for countries is called "Country" and not Nation"
like I previously thought"

   Args:
       text: The evidence text to store (e.g., "The foreign key in the 'sales' table
to the customer is called 'buyer_uuid'")
       database_id: The database identifier (e.g., 'formula_1', 'california_schools')

   Returns:
       Acknowledgment message.
   """
   # List of known/allowed database IDs
   known_databases = db_names

   if database_id not in known_databases:
       return f"Error: '{database_id}' is not a known database. Allowed databases: {',
'.join(known_databases)}"

   try:
       # Create a Document object with the evidence text
       doc = Document(
           page_content=text,
           metadata={
               "db_id": database_id,
               "source": "agent_learned_evidence"  # Tag to identify agent-learned
evidence

           }
       )

       # Add the document to the vector store
```

```
        vector_store.add_documents([doc])

        return f"Evidence successfully added to database '{database_id}'. Document
added with content: '{text[:100]}{'...' if len(text) > 100 else ''}'"

    except Exception as e:
        return f"Error adding evidence to vector store: {str(e)}"
```

Figure 4.10 shows the results of the two experiments:

1. In the first experiment, I used an approximately 40% sample of the BIRD benchmark. In the second, I took 10% of the benchmark (a subset of the previous 30%).
2. Using a second dataset, I asked another LLM (GPT-5 in this case) to generate three synthetically rephrased questions for each data point. I used the resulting dataset (close to the same size as simply taking 40% from the original) to test the agent over time.

    a. For example, one of the questions was "Where is Amy Firth's hometown?" The synthetic question came back as "What town does Amy Firth originally come from?" and "In which place did Amy Firth grow up?"

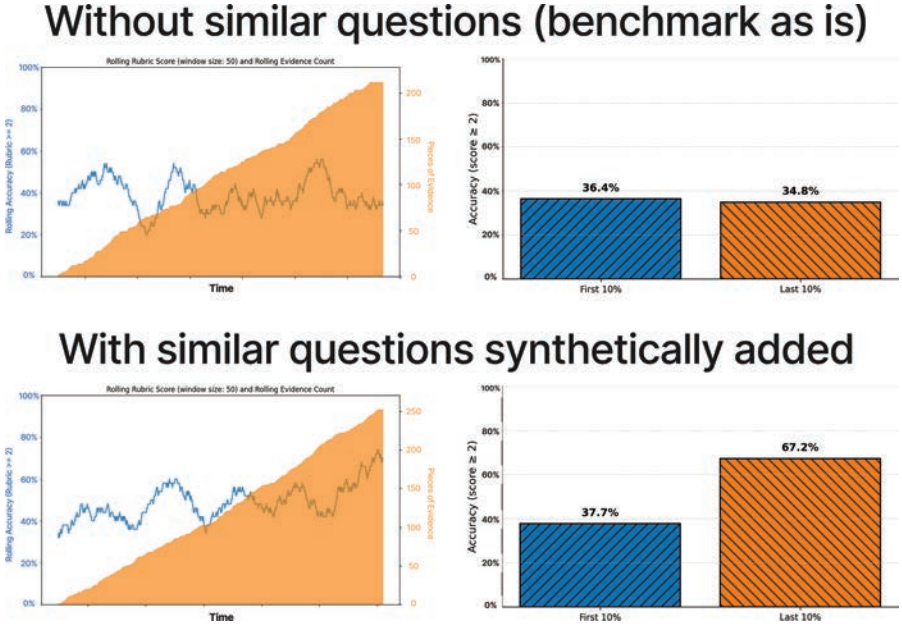


Figure 4.10 With no synthetically similar questions (top), we see no noticeable change in modified accuracy (rubric score ≥ 2). When we added in similar rephrased questions (bottom), suddenly the AI was allowed to rely more on its own written evidence as hints for how to tackle the problem.

I'll highlight a few points about the results:

- Without evidence, both agents started off with a notably worse accuracy than previously reported (approximately 36% here versus 51% in a previous section). This is evidence that the retrieved statements are crucial to the AI agent solving these questions correctly.
- In the case of the BIRD benchmark with no synthetic additions (the top two graphs), the AI agent is writing down information as it goes, but that information isn't useful to it in the future. If anything, this is a positive note for the benchmark itself: It implies that the questions in the benchmark do not overlap too much and are covering a wider span of cases efficiently. Basically, the questions aren't repeating themselves, which is what we want in a test set.
- When we added in the synthetically similar questions (the bottom two graphs), we see a big bump in accuracy over time, from 37.7% to 67.2% accuracy. Now the AI is adding information that becomes useful in the future. This accuracy bump doesn't appear immediately, because it took a while for the AI to write down enough useful information—but it did get there in the end.

  This is also a great jumping-off point for new experiments. For example, what if the AI were asked to "grade" logged evidence and was allowed to edit or delete evidence that didn't end up being useful? What if a second prompt/agent were in charge of retrieving evidence, and the tool our primary agent called was simply a proxy into a workflow that performed a more structured RAG to retrieve evidence? These are all great ideas to try against our test set.
- The number of times the agent decided to log evidence seemed to increase in the dataset with the number of synthetic questions added (as seen in the percentage of conversations using the `log_evidence` tool in Figure 4.11).

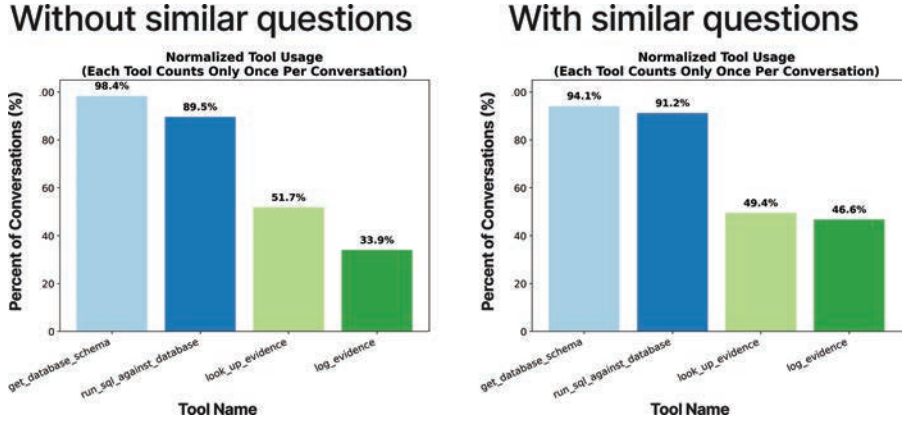


Figure 4.11 "Otto" (the agent with a notebook) called for the database schema fewer times and logged more evidence when the database included a larger number of synthetically similar questions.

Quite honestly, I don't know "why" the last result happened. My theory is that the AI agent noticed when it was looking up evidence that matched a synthetically generated similar question, which encouraged the AI agent to log even more evidence. (Note that in Figure 4.10, the raw number of evidence points added to the database was also higher in the synthetic dataset experiment.)

It's easy to see the differences between workflows and agents in specific use-cases like our SQL/RAG generation task. In general, though, you should take a few considerations into account when deciding whether to go with a workflow or an agent.

When Should You Use Workflows Versus Agents?

So, what's the takeaway from the last case study? Sure, you can just set an agent loose with some tools and let it figure things out for itself, but that's not usually the most efficient (or cheapest) approach. Here's how I would break it down:

- Workflows are the right move when you know the pathway ahead of time and don't expect many unforeseen edge cases. If you have a repeatable task and clear logic, just build the steps directly.
- Workflows require more upfront coding. You'll need to define all the steps, nodes, and edges; handle the branching logic; and think about edge cases yourself. But once it's built, the solution will be efficient.
- Workflows let you bake in efficiency tricks, such as optimizing for the number of examples to use in a few-shot prompt, quality checks, or early exits when a step fails.
- Agents are better at adapting on the fly. If your task isn't always the same, or if you want the system to "figure it out" and possibly learn new shortcuts or solutions, agents can do that. But it might take a while for them to get truly good at it, assuming the system even has the ability to do so (the `log_evidence` tool was created by me, not inherent to the LLM).
- Agents can easily switch between chatting and doing work. Need an AI system that can both explain something to a user and then go run a few database queries? Agents are built for this kind of flexible, multi-step interaction.

Bottom line: If you want pure efficiency and can define the process, go with a workflow. If you want flexibility and potential for the system to "learn" over time (and you're okay with some bumps along the way), let an agent loose with the right tools. When in doubt, *test, test, test!* Develop some hypotheses, set up a testing environment, and experiment to your heart's content.

We are just getting started with agents and our experiments with them. Next, we'll look at a tricky situation: What if we need multiple agents over a period of time to tackle a complex long-term goal?

Case Study 4: A (Nearly) End-to-End SDR

A **sales development representative (SDR)** is a sales professional (usually a human) focused on finding and qualifying potential customers for a company. SDRs are the initial point of contact for leads, engaging with them and determining whether they are a good fit for a product or service. SDRs nurture leads, often passing qualified ones to other sales team members for closing. In this case study, we will attempt to automate as much of this process as possible using real-world systems like Hubspot (a CRM), Resend (an email-sending service), Google (web search), and Firecrawl (a web-crawling API).

Agent 1: Lead Generation

The idea we'll use as a guiding star is to create an agent for each section of the SDR's job. Our first agent's job will be to go on the internet and find potentially good leads. All agents will have a generic prompt indicating how they are part of an SDR function. The lead generation agent is also given a specific prompt, as shown in Prompt 4.1.

PROMPT 4.1 Extra Information in the Lead Generation Agent Prompt

You are a lead generator. I need you to find professors/lecturers at universities who speak English and teach Data Science, ML, AI, etc. Leads you find should potentially be able to use my book in their classes. Find as much information as you can about the classes they teach, their research, and their publications. When you find a suitable lead, you must do the following:

1. Create a HubSpot contact for them to hold their email address.
2. Add a note with as much information as you can about the lead, including where you found their email, links to their research and publications, their personal website, and any other relevant information.
3. Add their email address to the contact.
4. Set their lead status to "New." Follow these steps in order, and do not skip any steps.

Prompt 4.1 gives the agent specific instructions in order and reminds it of things that a human might find obvious, such as following the steps in order and not skipping any. It essentially tells the agent to find people who might use my book (the standard prompt has basic information about my book in it) and record their information in the CRM.

Figure 4.12 visualizes this agent and three of the tools I would want it to potentially have. However, before giving the agent access to tools like web search, web crawling, and contact management in the CRM, we should consider an increasingly popular method for letting agents discover tools—MCP.

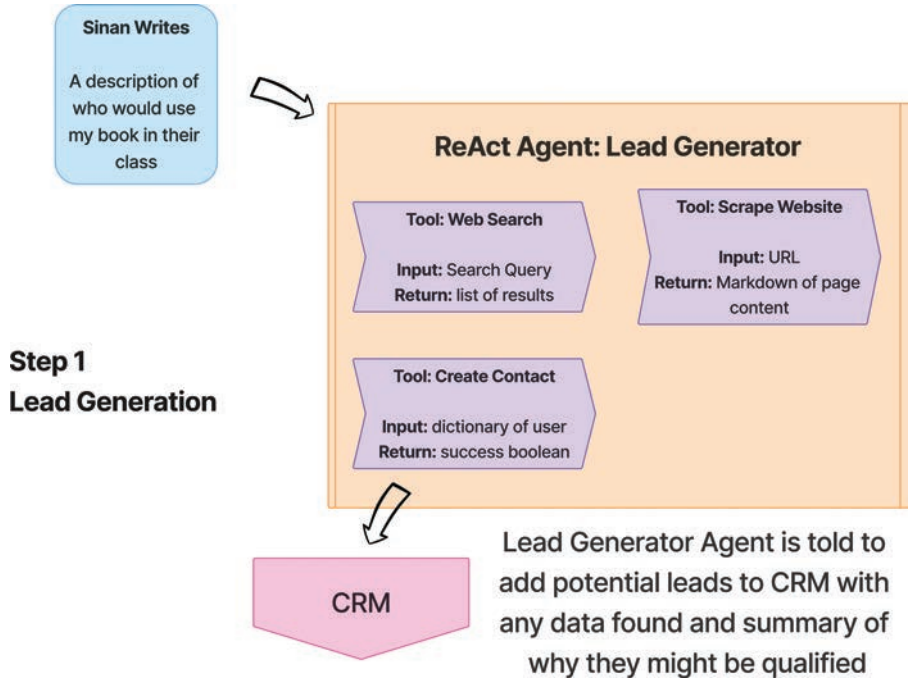


Figure 4.12 The lead generation agent is tasked with finding potentially qualified leads by using simple web crawling and web searching tools alongside tools for updating the CRM.

MCP for Flexible Tool Discovery

In November 2024, Anthropic released the **Model Context Protocol (MCP)** to not much initial fanfare. Anthropic was attempting to standardize the way we introduce tools (and other resources—but mostly tools) to AI models. The idea behind MCP (visualized in Figure 4.13) is actually quite simple: The MCP server is an API server (in Python, JavaScript, or some other language—it doesn't matter) that houses, among other things, definitions for tools and the capacity to execute the tools. The MCP server has API endpoints to list tools, execute tools, and so on. Put another way, MCP is a standardized API format placed in front of the tool definitions and execution code that makes it easier to develop agents across frameworks, programming languages, and businesses.

When an AI agent "wakes up," it reaches out to the MCP servers included in a list that it was given and asks each of those servers which tools it can offer. Each MCP server then tells the agents the names of the functions and the arguments they take. The code to actually execute the code lives on the server, too. Once the AI agent grabs the tool definitions from the server, everything is effectively the same to the LLM—calling tools, writing content, and so on.

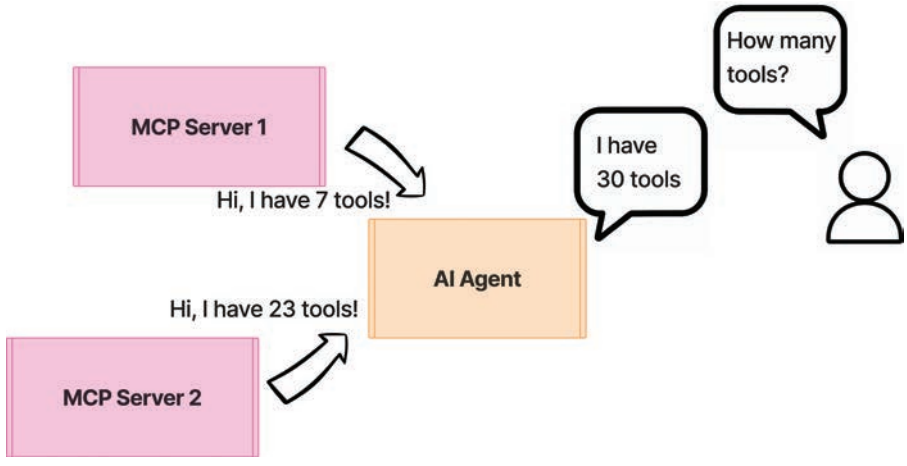


Figure 4.13 An agent is told about one or more MCP servers. On initialization, the agent reaches out to each one and asks which tools it has for it to use. From there, everything is the same as far as the LLM is concerned. It has tools and functions it can decide to call, no matter where they came from.

In summary, MCP is an agreed-upon standard that, if everyone follows it, can facilitate the development of tools for AI systems around the world. A developer in California can write an MCP server for a tool that they made (or didn't!) and put it on GitHub, where a developer in Mumbai can find it and use it. In fact, in this case study, I will write just the first two MCP servers; the third one was created by a third-party team where I had no involvement. I just got to use it for free—how amazing!

There are caveats to MCP, of course. Clearly, tool design and selection are critical, as we saw with our SQL agent. Using the right tool can be efficient and is often a necessary step. Because MCP servers give an AI model access to a set of unknown external tools, however, the challenges will be compounded. The agent will inevitably encounter new tools with descriptions of varying quality. We can mitigate this risk by making sure the descriptions we write for our tools are sufficiently contextful, but even a single bad tool description can send agents down a completely wrong path.

With that warning, let's define our first two MCP servers. These are the MCP servers that I wrote specifically for this case study.

MCP 1: Web Search + Web Crawling

The Python template for creating an MCP server generally has three parts:

- An API route to list the tools available on the server (names, arguments, etc.)
- An API route to accept arguments to execute a tool call
- Logic to call and execute each tool

Listing 4.5 shows an abbreviated code snippet of our first MCP server delivering the ability to Google something (using the Serp API product) and to crawl most websites on the planet (using Firecrawl's web-crawling service as an API to make this easier). Note that we are using the official Python MCP implementation here: https://github.com/modelcontextprotocol/python-sdk. Using this official implementation means that we just have to write the Python functions correlating to listing tools, calling and executing tools, and so on.

Listing 4.5 **Creating the first agent in LangGraph**

```
from mcp.server.models import InitializationOptions
from mcp.server import NotificationOptions, Server
from mcp.types import Tool, TextContent
import mcp.types as types ...

# Import required libraries
from serpapi import GoogleSearch
from firecrawl import FirecrawlApp
# Create server instance
server = Server("research-mcp-server")

def search_with_serpapi(query: str) -> str:
   """Search the web for the query using SerpApi."""
   api_key = os.environ.get("SERP_API_KEY")
   if not api_key:
       return "Error: SERP_API_KEY environment variable is required"

   try:
       search = GoogleSearch({
           "q": query,
           "api_key": api_key,
      ...
      return "\n".join(formatted_results)

   except Exception as e:
       return f"Error performing search: {str(e)}"

def scrape_with_firecrawl(url: str, format_type: str = "markdown") -> str:
   """Scrape a webpage using Firecrawl."""
   api_key = os.environ.get("FIRECRAWL_API_KEY")
   if not api_key:
       return "Error: FIRECRAWL_API_KEY environment variable is required"

   if format_type not in ["markdown", "links"]:
       format_type = "markdown"

   try:
       firecrawl = FirecrawlApp(api_key=api_key)
```

```
        ...
        return response if response else "No content found"

    except Exception as e:
        return f"Error scraping URL: {str(e)}"

@server.list_tools()
async def handle_list_tools() -> List[Tool]:
    """List available research tools."""
    tools = []
   tools.append(Tool(
        name="web_search",
        description="Search the web for information using SerpApi. Returns top 3 search
results with titles, snippets, and URLs.",
        inputSchema={
            "type": "object",
               ...
    return tools

@server.call_tool()
async def handle_call_tool(name: str, arguments: Dict[str, Any]) -> List[types.
TextContent]:
    """Handle tool calls for research operations."""

    if name == "web_search":
        query = arguments.get("query")
        if not query:
            return [types.TextContent(type="text", text="Error: Query is required")]
        result = search_with_serpapi(query)
        return [types.TextContent(type="text", text=result)]

    elif name == "scrape_website":
        url = arguments.get("url")
        format_type = arguments.get("format", "markdown")
        result = scrape_with_firecrawl(url, format_type)
        return [types.TextContent(type="text", text=result)]

    else:
        return [types.TextContent(type="text", text=f"Unknown tool: {name}")]

async def main():
    ... code to actually run the server

if __name__ == "__main__":
    asyncio.run(main())
```

That was a lot of code, but don't worry: I won't be showing much more MCP code from here on out (it's all in the book's GitHub). In Listing 4.5, you can see the three main parts there: functions to run each of our two tools, a route to list tools, and a

route to accept arguments to execute a tool. With that, we have our first MCP server, giving an AI agent the ability to look things up and visit web pages.

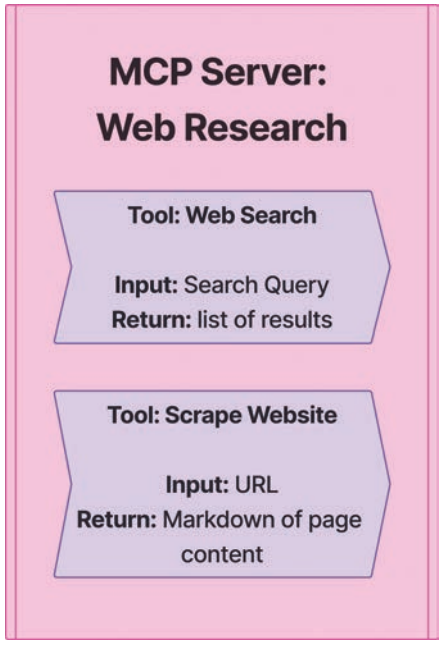


Figure 4.14 Our simple homegrown MCP server has only two tools: web search via the Serp API (Googling something as an API) and web crawling (a service provided by Firecrawl).

Figure 4.14 visualizes our first MCP server with two tools. A lead generation tool will certainly need to look things up and visit web pages. It will also need a system of record to write everything down in.

MCP 2: Hubspot Management

This isn't our first discussion about a system of record for an AI agent. In Chapter 3, we watched an AI system improve at a task over time after we gave it a notepad to write things down in. In this case, we don't really need the agent to get better over time (at least for now), but we should be able to audit what is going on in the CRM. I chose Hubspot for this purpose (feel free to choose a different CRM) mostly because the APIs were easy to pick up. Figure 4.15 shows the five tools I wrote to connect to the Hubspot API:

- **Create contact:** Create a new contact in the CRM.
- **Update contact:** Update an existing contact.
- **Add note to contact:** Add a free text note to a contact.
- **Retrieve notes for contact:** List all notes for a given contact.
- **Fetch contacts:** Use search criteria to grab lists of contacts.

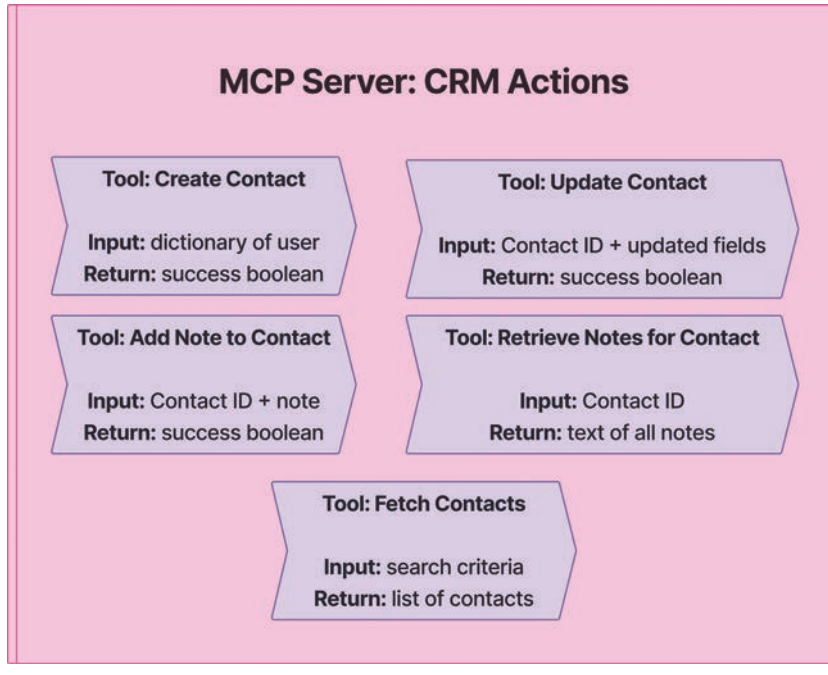


Figure 4.15 Our more complicated homegrown MCP server has five tools for interacting with the CRM so the agents can keep us in the loop about their work.

I did try to find a Hubspot MCP server on the internet, but the few I found either didn't work as written or didn't have the tools I needed (in particular, the ability to create a contact). So, in this case, I asked an AI agent to write an MCP server for me given the first MCP I wrote as an example—and it worked great!

Now we have an agent with seven tools, and Figure 4.16 shows the current state of our lead generator agent. Now, we need to qualify these leads.

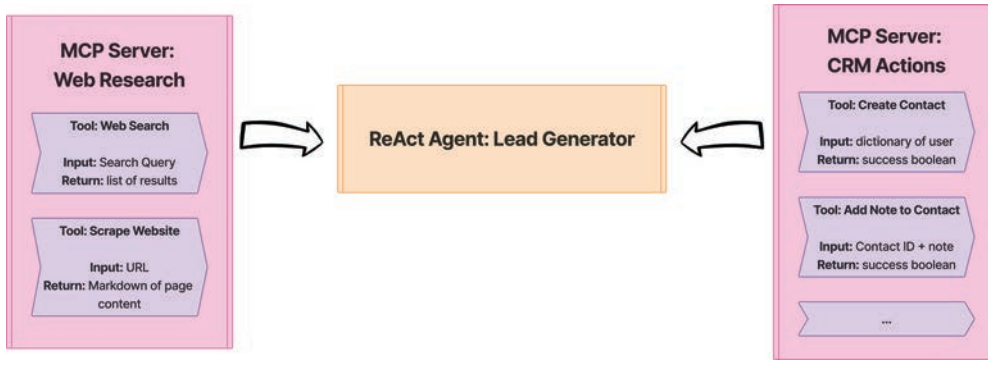


Figure 4.16 The Lead Generator agent gets its seven tools from two MCP servers.

Agent 2: Lead Qualification

Once the lead generation agent adds a new contact to the CRM, the lead qualifying agent (pictured in Figure 4.17) will take over. It will both double-check the work of the lead generation agent and check some more criteria that I added into the special prompt area. At a glance, it might seem as if the lead qualifier agent is redundant given the lead generation agent. They have the same tools and basically the same goal (to identify qualified leads), but the key difference lies in the context of the agent. Chapter 7 formally dives into the idea of **context engineering**—the concept of providing an AI model or agent with the necessary information, context, and tools to perform a task effectively.

The lead generator is given a small set of rules by which to judge candidates and is asked to pull potential leads from the wide world of the open internet. In contrast, the qualifier agent is given both a single person's information and a longer set of requirements to check before moving on to the next stage. In other words, the lead generator has the easier job of pointing a finger at someone and saying, "They seem right"—a task that can be performed by a smaller, cheaper, faster LLM. The cost of a false positive in this case is low because we know a second agent will double-check its results. The lead qualifier agent should be a larger, slower, more expensive LLM because its job is arguably harder: It must go through several pieces of information; read and understand the candidate's syllabus, CV, and other data; and make the final judgment whether the person should receive an email. The cost of a false positive is high here, because I don't want to bother people who aren't good fits for my book.

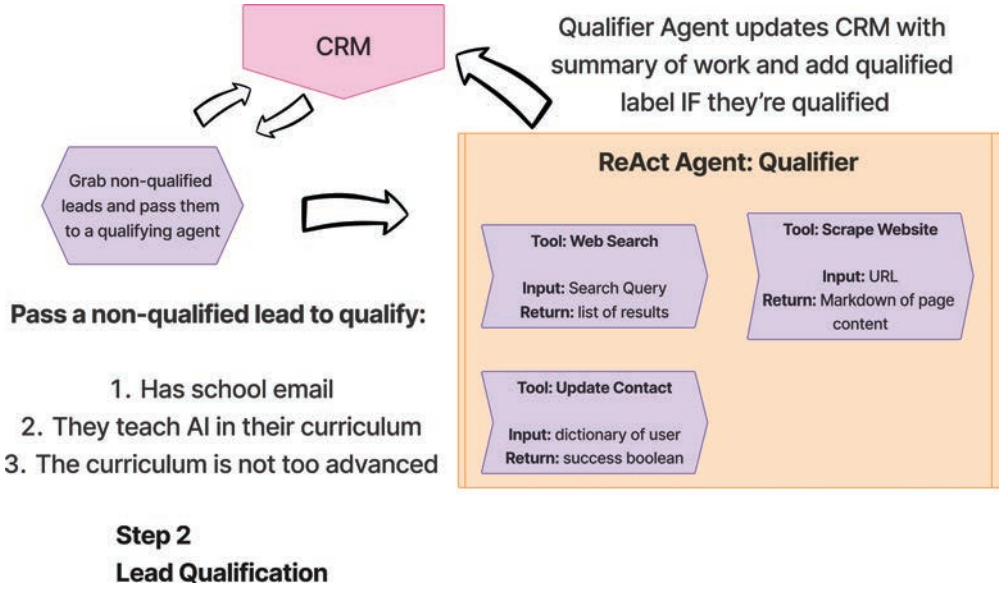


Figure 4.17 Our lead qualifying agent will have the same tools as the lead generation agent. It will double-check the lead generator's work while also doing more research.

Like the lead generation agent, the qualifying agent will have some extra information in its system prompt. A snippet of this information can be seen in Prompt 4.2.

PROMPT 4.2 Extra Information in the Lead Qualification Agent Prompt

You are a lead qualifying agent. For a given lead, use the tools provided to make sure that this lead is a good fit for using my book in their classes. If they are a good fit, update the contact to have a lead status of "Open." If they are not a good fit, update the contact to have a lead status of "Unqualified."

Below is a list of qualities that the person must match before marking them as "qualified":

1. They must be teaching in an accredited university or college setting.
2. A recent syllabus for a class they currently teach must reference large language models (LLMs) in some capacity.

...

Figure 4.18 shows that this agent will have the same MCP server access as the lead generation agent. In some cases, it might need to double-check some information online and update information/notes.

At this point, the lead generator has pulled in some leads, and the lead qualifier has double-checked the information and marked the lead as qualifying. Now it's time for a third agent to take over and send the initial cold email.

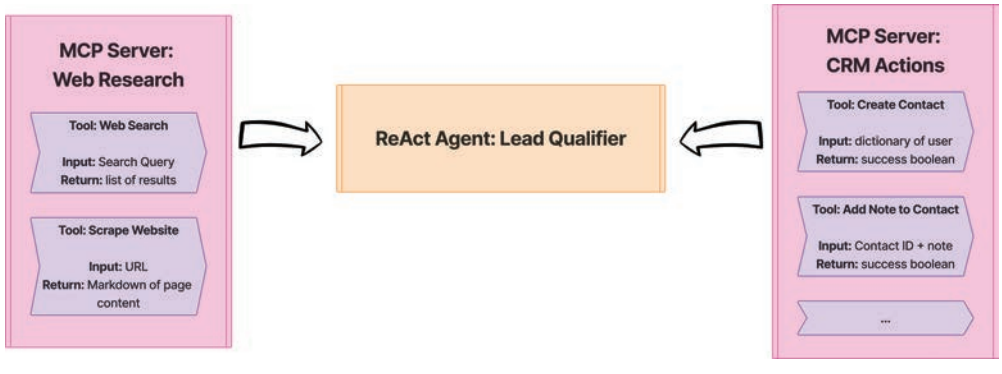


Figure 4.18 Our lead qualifier has the same MCP servers as the lead generator. Its job is to do deeper research on the leads and make sure they are good candidates.

Agent 3: Lead Emailing

The lead emailing agent (Figure 4.19) involves more than just a system prompt change, with a single-shot example (recall from Chapter 1 that a *k*-shot/few-shot prompt means I am placing in-context examples in the prompt to guide the AI) showing how I want the email to roughly sound (as seen in Prompt 4.3). The format of the email in the single-shot email is not as important here because we expect the agent to use a tool via MCP with structured inputs. What's more important is the fact that it writes HTML-encoded emails and subject lines and sends them to the right tool.

PROMPT 4.3 Extra Information in the Lead Emailing Agent Prompt

You are in charge of sending an initial outreach email to a lead. Use the tools provided to look up the information gathered by the other agents, and send an email to the lead with a personalized message. The email should be sent to the lead's email address. When you are done, update the contact to have a lead status of "Connected" so we know that you have sent the email. Here is a sample email:

Subject: "Curious If 'Quick Start Guide to LLMs' Could Be a Fit for Your Course?"

HTML Body: Hi [Name / Dr.]! My name is Sinan Ozdemir. I'm an AI author, educator, and the . . .

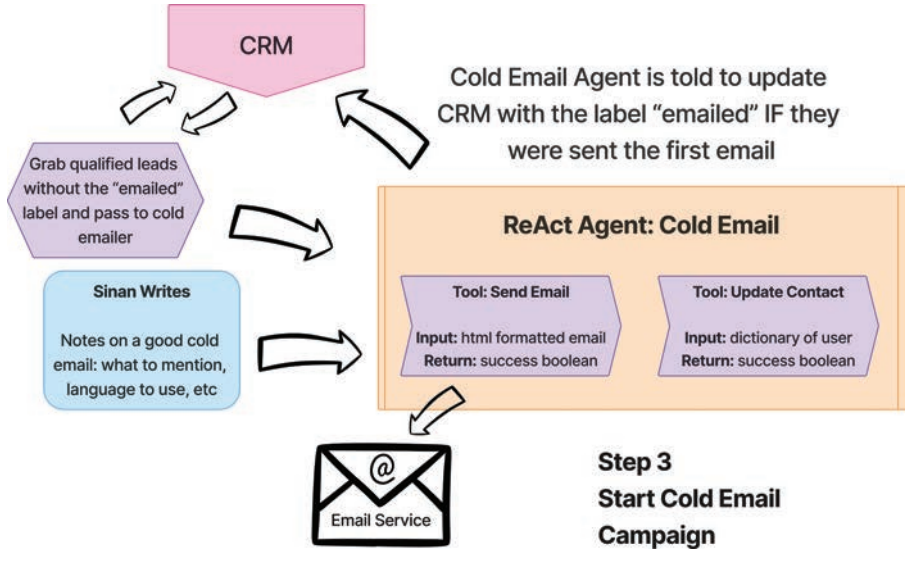


Figure 4.19 The lead emailing agent needs to update the CRM, but also requires new capabilities to email leads on my behalf.

A quick note on Agent 3: A decent argument can be made for making this final agent a workflow. For one thing, Agent 3 isn't given much agency. We know the lead is qualified given the previous two agents' work, and all this agent has to do is write the email and call a few APIs in order—that sounds like a predefined pathway. The reason I wanted to make this a true agent over a workflow was simple: I assume this task is easy enough that a system prompt with clear instructions is enough for the agent to follow my instructions clearly. In Chapter 5, we will start to put some of these assumptions to the test. For now, by choosing an agent over a workflow, we are effectively saying either "This task requires the agent to make decisions on the fly that are too complicated/near impossible to code in a predefined pathway" or "This task is easy and straightforward enough that we can save development cycles by simply attaching preexisting MCP servers to an LLM with proper context."

Assuming this job is left up to an agent and not a workflow, this agent will need a way to send emails. MCP comes to the rescue once more.

MCP 3: Resend for Email

If you're not familiar with it, Resend is a developer-friendly email platform with APIs to simplify sending emails at scale. What's even cooler is that it has an official MCP server on its GitHub. Granted, at the time of writing there is just a single tool on that MCP server, but it's the tool I most care about: send-email. I could write my own MCP server for Resend, just as I did with the CRM. However, I want to showcase that our system is agnostic regarding who writes MCP servers and in which language the code launches the official Resend MCP server (written in Typescript, not Python). Our agent will be able to use both it and our homegrown Pythonic MCP servers.

Figure 4.20 shows the topline view of our third agent. The web research MCP has been replaced (because in theory it won't be needed because of the past two agents' work) with our new MCP server, and it is ready to send emails.



Figure 4.20 Our final MCP server has a single tool and was developed by the same team that created the email sending service Resend.

When to Use a Multi-Agent Versus a Single Agent

Why didn't we just create a single agent with an elongated set of instructions to walk through the whole process of generation, qualifying, and emailing? Certainly, we could have: We could just make that agent wake up and attempt to find and email a new lead. In theory, we could spin up a hundred of them, giving each a different location and industry to focus on.

I chose a multi-agent system in this case study for the following reasons:

- To reduce potential errors in which the agent emailed someone before qualifying them and adding notes to the CRM. In other words, what if the agent "forgets a step" along the way?
- To minimize overlap in duties. What if two agents that aren't talking to each other or are in some race condition end up trying to email the same person at the same time? That lead would not appreciate the double email and would probably be lost.
- To leave room for experimenting with different flavors of LLMs (including LLMs fine-tuned for specific tasks) for each of the different tasks. Maybe GPT-5 is slower but better at emailing, whereas Llama 4-Scout is optimal for quicker lead generation.

It's absolutely true that these tasks *could* be handled by a single agent, but by splitting them up, I gain more control over specific aspects of the funnel. This way, if qualifying is going poorly but everything else is okay, I can tweak and experiment on qualification alone without too much risk to the other portions of the pipeline. With a single agent, every prompt change risks a regression in another area. For this reason, a fair analogy to multi-agent systems would be a micro-service architecture. A change to a single service can be done in much more isolation as long as we are aware of how that single service (agent) fits into the larger picture.

We have now designed and coded three agents, each handling a different portion of the sales process (Figure 4.21). However, we lack a mechanism to test this engine or to keep it running, constantly finding new leads, qualifying them, and emailing them. We will tackle the latter issue in Chapter 5. To gut check the functionality of our agents, though, we should chat with them to see how they do.

Streamlit for Ad-Hoc Testing of the Three Agents

In the codebase for the multi-agent system, I created a visual interface where you can select one of the three agents and chat with it, asking it to perform tasks on demand. Figure 4.22 shows an example in which I asked an agent to list contacts in the CRM and it did so correctly (displaying the two fake contacts Hubspot created during account creation).

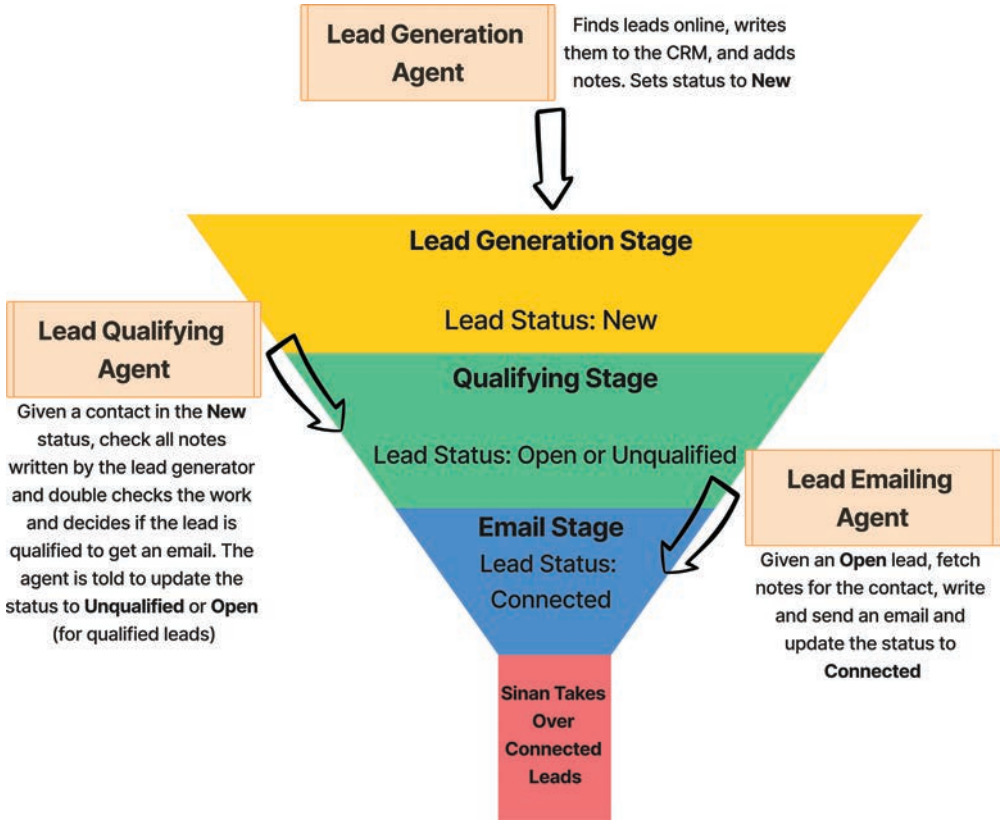


Figure 4.21 The 10,000-foot view of our multi-agent task delegation. The lead generator will fill the top of the funnel, the qualifying agent will get enough information to be able to email qualified leads, and the emailing agent will send off that first cold email on my behalf.

When I asked the lead generation agent to find a contact at a specific business school, it did. (This example isn't shown here because it would invade that person's privacy.) When I asked the qualifying agent to run on that contact, it changed the status and added a note to confirm the contact's qualification. When I asked the email sending agent to send an email, it did; it then changed the contact's status again, as expected.

At this point, I'm satisfied with the agents' individual performances from basic ad-hoc testing. However, if we are to trust this system at scale, we need to consider a few more factors.

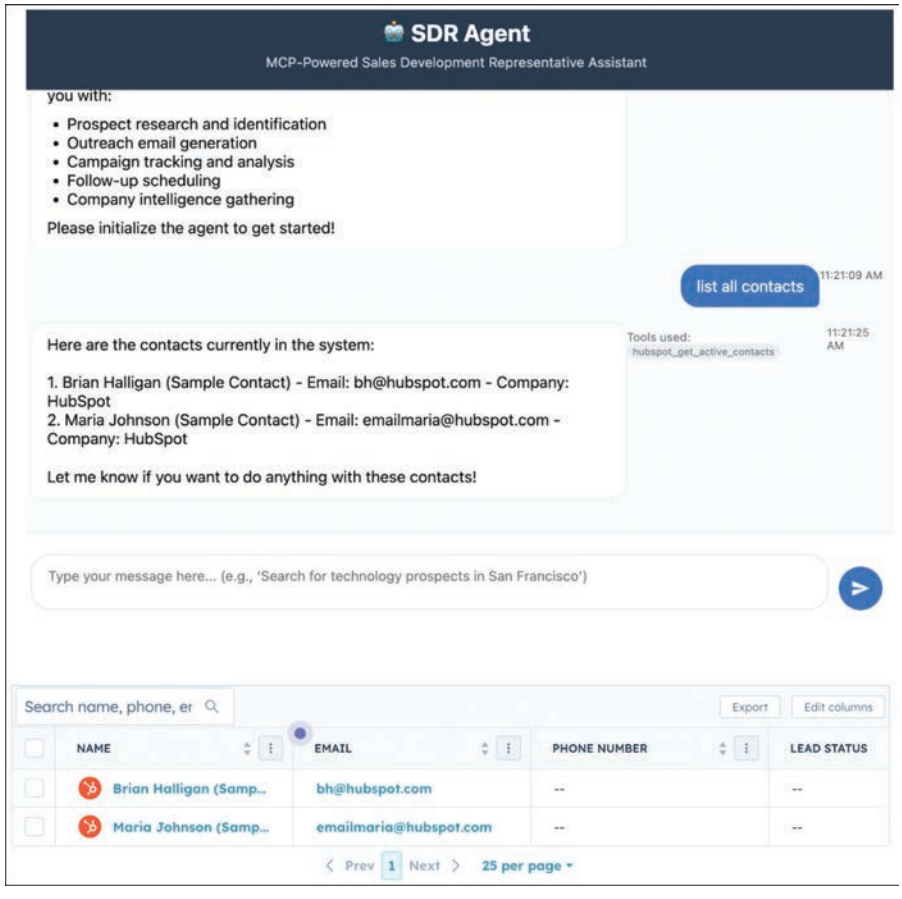


Figure 4.22 A Streamlit app (found in the book's GitHub) showing basic capabilities of the agents for interacting with the CRM.

Evaluating Agents

We've already discussed ways to evaluate agents, including latency, cost, and memory considerations, and especially in comparison to rigid workflows. Figure 4.23 breaks down even more ways to consider evaluating agents in four main categories:

- **System:** Focused on the "behind the scenes" functionality of an agent (e.g., the tool latency, the LLM provider error rate).
- **Quality assurance:** Judging the LLM's ability to adhere to instructions and follow a specific output format. The quality of that output falls into this category as well.

- **Tool interaction:** Specifically zooming in on the agent's tool-calling ability, including whether the agent is calling the right tools for the right tasks and whether it passed the right arguments into the tool.
- **Agent efficiency:** Dependent on the tool interaction and system results to a degree. This category is concerned with questions like how many steps the agent took to come up with the answer, how many tokens it took, and how much it cost.
  - For example, an agent that provides 1,000 tokens of reasoning just to call a single tool could be slower and more expensive than an agent that calls five tools with no thinking in between, even though the latter would be considered "tool inefficient" compared to the former's "token inefficiency."

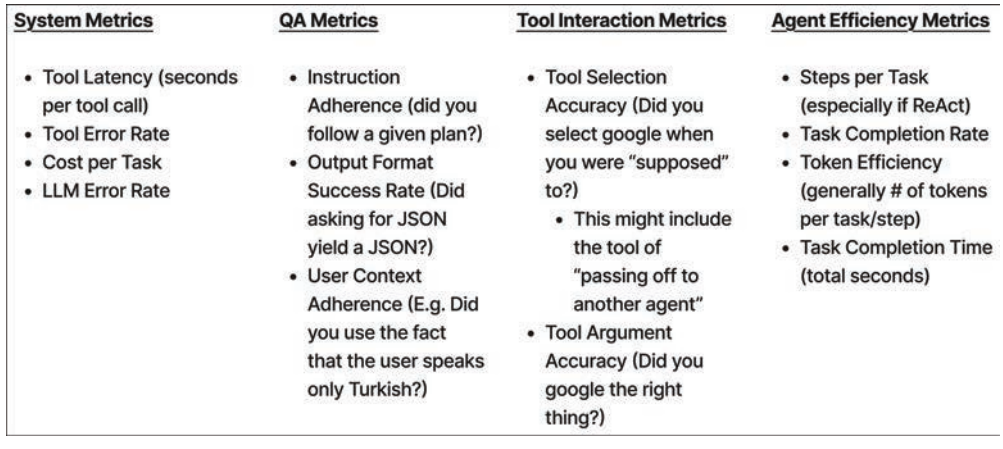


Figure 4.23 There are dozens, if not hundreds, of metrics we can use to measure the efficacy of agents and multi-agent systems, ranging from low-level tool-based metrics to holistic rubric-based accuracy.

We can also rely on third-party tooling (no pun intended) to help us audit and look back on our agents' past work.

LangSmith for Traceability

Even if all we need is a way to audit our agents, there's no reason to reinvent the wheel. **LangSmith** (brought to us by the same people who developed LangGraph) is a tracing and observability tool that is purpose-built for LLM workflows. It can track the step-by-step execution of both workflows and agents. LangSmith provides a live dashboard showing every input, output, and tool call, which makes debugging and long-term monitoring a breeze. It also give us the ability to audit past work. There are dozens (at least) of platforms offering observability, evaluations, and more, but LangSmith is free to get started with, simple to set up, and provides immediate value out of the gate. I generally recommend starting with LangSmith even if you are still evaluating other

platforms, as it provides an excellent baseline for bare-bones features you should expect from other platforms.

Getting started with LangSmith is a very simple process. All we need to do is set a few environment variables in whatever deployment we are using (even a code notebook), and the SDK will automatically trace everything (see Listing 4.6). We can even split traces by project or use the built-in UI to review results across experiments.

Listing 4.6 **Setting LangSmith keys in the agent environment**

```
- LANGSMITH_API_KEY="XXX"
- LANGSMITH_TRACING=true
- LANGSMITH_ENDPOINT="https://api.smith.langchain.com"
- LANGSMITH_API_KEY="XXX"
- LANGSMITH_PROJECT="ai-agent"
```

Once the environment variables are set, every agent run (including tool calls, LLM generations, and custom functions) will be recorded in a LangSmith dashboard. This lets us dig into failed runs, analyze tool usage, and share and trace URLs for easier debugging and collaboration. Figure 4.24 shows what an agent trace looks like in the LangSmith user interface.

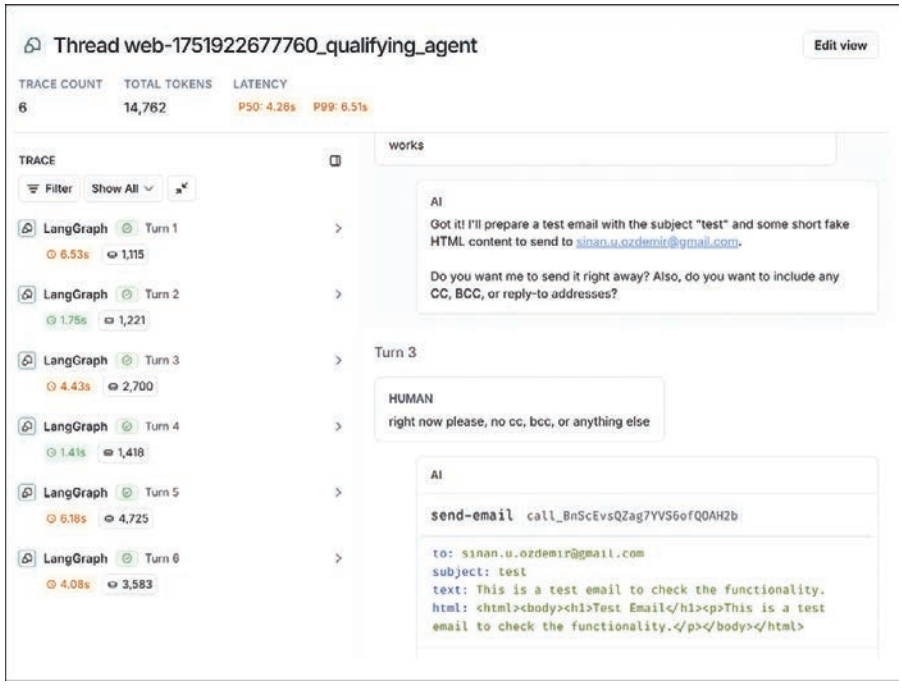


Figure 4.24 An agent trace in the LangSmith dashboard, highlighting each tool call and LLM step. LangSmith offers an easy way to get auditable logs by adding only a few environment variables to the AI system.

In the next few chapters, we will continue to evaluate agents based on the specific work they're asked to do. But for now, let's wrap up this Agents 101 discussion with a recap.

Conclusion

The leap from workflows to agents to multi-agent systems can unlock a whole new set of capabilities for AI applications. Workflows are great choices when you want efficiency, predictability, and full control. Agents bring adaptability, flexibility, and a little bit of creative chaos, which can be a wonderful thing in some cases.

In this chapter, we broke down not just how to build these systems, but also how to evaluate them using both human-driven rubrics and automated tools like LangSmith for traceability and auditability. Although agents can sometimes cost more and take longer, they also offer room for learning, collaboration, and improvement over time. That is especially the case when the agent is given the ability to write, recall, and share evidence with itself (recall our "Otto" example) or with other agents (as in the SDR framework).

Bottom line: There's no single "best" approach to developing systems with AI agents. The right choice depends on your use-case, your tolerance for ambiguity, and how much control you want versus how much flexibility you need. Try both approaches. Experiment, evaluate, and build a system that works for you, and don't be afraid to combine the best parts of each.

In Chapter 5, we'll do just that. We'll create agentic/workflow hybrids and then take these ideas even further, exploring long-running, multi-agent workflows that can handle complex, evolving tasks over time. See you there!

Index

A

accuracy
  chain of thought to increase, 67–68
  ECE to predict, 232–233, 237
  as LLM comparison factor, 63–65
  measuring agent, 95
  and non-use of tools, 132
  privacy/cost/speed as compared to, 242–243
  quantization and loss of, 263, 265
  state of the art, 63, 65
  testing agentic tool use for, 152–156
  testing retriever, 70
  workflow vs. agent, 99
action, reasoning before, 27
ad-hoc testing, 116–118
agency, agent, 28
agentic workflows, 133–141
agents
  agentic vs. workflow cost, 94–95
  agentic workflows, 133–141
  ALWAYS USE prompt for, 130
  clarification requests from, 155
  coding agents, 176–188
  converting SQL workflow to, 87–104
  decision-making by, 88, 95
  defined, 87
  end-to-end SDR, 105–118
  enhancing behavior of, 123
  evaluation of, 95–100, 118–121, 131–133
  evolving systems to, 45
  extended mind systems for, 10–103
  flexibility of, 133
  multi-agent architectures, 141–147
  multi vs. single, 116–118
  overview of, 25–28
  policy bot agent, 127–133
  policy compliance agent, 124–127
  prompt engineering, 130
  reasoning models for, 209–212
  reflection by, 135–136
  rubric for grading, 96–99, 205
  separate modality handling by, 160, 168
  source of failures, 95
  supervisor agent, 148
  testing agentic tool use, 149–156
  tool decisions by, 25, 88, 130–133, 197
  tracing execution of, 119
  using multiple, for complex goals, 104
  workflow vs. agent use, 88, 99–100, 104, 115
AGI (artificial general intelligence), 195
AI
  agents, 25–28
  AI calibration, 132
  computer use by, 212–213
  evaluation of, 59
  experiments with noticeable outliers, 155–156
  as extended mind, 100
  increasing system reasoning, 195
  limits of text-only, 159
  rewards from experimenting with, 284
  seven pillars of intelligence for, 195–196
  Transformer architecture for, 6

See also large language models (LLMs)
alignment, 10–12
ALWAYS USE tool prompt, 130, 132
Anthropic, 21, 63, 106, 179, 187, 198, 199, 205, 206, 217
any-to-any models
  architecture for, 141, 142
  multimodal, 188–190
API servers, 106, 176, 235
app_reviews task, 229–231, 234, 240
A Quick Start Guide to Large Language Models (Ozdemir), 4, 159, 166, 205, 227, 247, 248, 262, 266
architecture
  LLM, 3
  multi-agent, 140, 141–147
  RAG, 31, 48
  supervisor, 141, 142, 148–149
  Transformer, 6
  Vision Transformer (ViT), 161
arguments, 93, 94
Aristotle, 133, 198
asks, 12
assistant models, 269
audio assistants, 165
augmenting prompts, 32
autoencoding models, 5–6, 69, 225, 228, 230, 237
AutoModelForSequenceClassification, 236
autonomous decision-making agents, 3
autonomy, agent, 26, 142
autoregressive LLMs, 5–6, 69, 156, 162, 164, 178, 198, 201, 230, 237, 246, 257

B

base and assistant models, 269
behavior alignment, 11
benchmarking reasoning models, 198–209
benchmarking STT and TTS models, 261
benchmarks, unstructured, 124–125
BERT, 164, 236, 237, 266, 282
bias, positional, 9–10, 156
BIRD benchmark, 33–34, 39, 75, 101, 102–103
BM25 (Best Matching 25), 127, 128

C

caching prompts, 19–23
calibration, AI, 132, 225, 231–238, 243–245
calibration curves, 234, 236–238, 243
centroid score, 218
chaining prompts, 16–17, 32, 80–83
chain of thought
  discursive reasoning and, 133, 198
  in few-shot learning, 65–66
  increased accuracy via, 67–68
  in non-reasoning models, 203
  positional bias and, 156
  for prompt engineering, 197
  in reasoning models, 14
  and seven pillars of intelligence, 196
Chalmers, David, 100
chat alignment, 11
checkpointer, 90, 127
Chroma database, 39
chunking documents, 247, 248–255, 258, 277
Clark, Andy, 100
classification tasks, 60, 226, 228, 230, 232, 241, 243
classifier engines, 228
Claude, 12, 187, 200
Claude 4 Sonnet, 14, 63, 81, 199, 206, 208-209, 217, 219
Claude Opus 4, 208-209
Claude Sonnet 3.5, 212
CLIP (Contrastive Language- Image Pre-training), 160–164, 166, 172
closed source, 69, 235
CodeAct, 178
coding agents, 176–188
Cohere, 69
Cohere V4, 71, 73, 75
Cohere VE, 76
Command object, 144

compounding failures, 26
compression of models, 261–268
computer use
  benchmarking with reasoning models, 215–220
  LangGraph to build, 220–223
  multimodal vs. grounded, 213–215
  by single LLM, 212–213
concatenation mechanism, 174
conditional edge, 55–56
confidence in model accuracy, 232–233, 244, 245, 257, 284
constrained decoding, 179
content engineering, 196
context engineering, 112–113, 197–198
context windows, 3, 7–10, 230
contrastive loss, 162
conversational system
  as agentic component, 25
  coding agents in, 178
  handoffs in, 144–147
  network-based architecture, 143
  storing histories, 55
conversations, 91, 94–95, 253–255
convolutional neural networks (CNNs), 160
coordinate tasks, 215–216, 218
cosine similarity, 43, 61, 68, 69, 77–80, 129, 162–163, 173
cost
  accuracy/privacy/speed as compared to, 242–243
  of chaining prompts, 82
  of fine-tuning, 238–242
  fine-tuning to reduce, 225
  as LLM comparison factor, 63–64
  prompt caching to control, 19, 21–23, 29
  prompt order and, 13
  of workflows vs. agents, 94–95, 99
CRM, 105, 106, 110–111, 114, 115
cron job, 148
customer support bots, 124

D

databases
  evaluating difficulty of, 75–77
  running SQL against, 91–92, 94
  size and query generation, 36
data generation, synthetic, 124–127
data input
  analyzing samples for, 35
  and context window size, 7–8
  database size, 36
  mixed types of, 188–190
  to nodes, 45
  processing times for, 19, 21
data privacy, 242
data sources
  evaluating summaries of, 77, 79
  predicting choice of, 77
  synthetic data creation, 125–126, 129
decoding, speculative, 269–272
Deep Research system, 133, 136–141
DeepSeek, 138, 198
denoising function, 164, 165
describe_database function, 35
de-serializing threads, 55
determinism, inference parameters and, 19
deterministic predefined workflows, 3, 28
diffusion models, 164, 165, 178
discursive reasoning, 133, 198, 202, 217
DistilBERT, 266, 268
distillation, 138, 266–268
documentation caching, 19
documentation/in-context learning, 12, 14
document length normalization, 127
domain adaptation, 226, 245–258
dot product, 43
dynamic fusion, 188
dynamic prompts, 13

E

edges, workflow
  conditional, 55–56
  and nodes, 38
  nodes combined via, 50
efficiency evaluation, 119
emailing leads, 114–115, 117
embedders
  domain difficulty and choice of, 75–77
  embedding tasks, 60, 61
  fine-tuning of, 77
  in semantic few-shot learning, 65
  testing, 45, 68–75
embedding_learning_rate, 256
embedding modalities in single vector space, 160–164
embedding models, fine-tuning, 277–284
embeddings, Matryoshka, 279–281
embedding similarity
  BM25 to replace, 129
  CLIP to maximize, 162–163
  document retrieval via, 58
  in evaluating retrieval, 68
  in multimodal image retrieval, 169
  to simplify workflows, 43
  testing embedders, 45, 69–75
end-to-end RAG system, 31, 32
environment, 25, 119–120
errors, 94, 178
Euclidean distance, 43
evaluation
  of agent response quality, 131–133
  of agents, 118–121
  of AI, 59
  of AI summarizations, 77–83
  of domain-adapted LLM, 257–258
  of domain difficulty, 75–77
  downstream effects of, 72
  of evidence retrieval, 68–75
  LLM comparison for SQL generation, 62–65
  noticeable outliers and flaws in, 155–156
  of policy compliance agent, 124
  pre-fine-tuning baseline, 234–235
  requirements for, 60
  via testing, 59
evidence
  database size, 36
  evaluating retrieval of, 68–75
  example of indexing evidence, 39–41
  grading/eliminating, 103
  lack of need for, 45
  retrieval of, 42–45
  in text-to-SQL workflow, 34
  tools to look up, 91
execute_code node, 178, 183
execution, SQL, 48–52
expected calibration error (ECE), 232–233, 243
experimentation
  to choose LLM type, 62–65
  in few-shot learning types, 65–68
  rewards from, 284
extended mind systems, 100

F

FAQ agent, 124
few-shot learning, 14–15, 32, 65–68, 114, 229
fine-tuning
  baseline evaluation prior to, 234–235
  for calibrated performance, 235–238
  data used for, 226
  embedding models, 277–284
  generative models, 245–257
  prompts to mimic, 228
  Qwen3, 252–257
  reasons for, 225
  and speculative decoding, 269
Firecrawl tool, 153, 154
float (temperature), 17, 18, 19, 46
Flux, 184
foreign keys, 36
formatting strings, 49
framework, common workflow, 39

free response tasks, 60, 61, 62, 243–245
frontier, 199
Frontier Labs, 152, 156
function calling, 24–25

G

gates, LLM, 77
Gemini, 189, 217
Gemini 2.5, 152
Gemini 2.5 Flash, 151
Gemini 2.5 Flash Lite, 9, 137
Gemini 2.5 Pro, 63
generate node, 178
generative models, 59, 164, 228, 245–257
generator
  calibration of, 244
  in RAG systems, 31
  in text-to-SQL workflow, 46–48
get_database_ schema, 90–94
Google, 63, 137
Google Brain, 161
googling terms, 153
GPT, 11
GPT-2, 162
GPT-4o, 189, 201
GPT-4o-Mini, 63, 65, 68
GPT-4.1, 130–133, 137, 140, 152, 189, 235, 237, 238–243
GPT-4.1-Mini, 62, 63, 81, 137, 140
GPT-4.1-Nano, 97, 130–133, 151, 152, 153, 229, 234, 239–243, 257, 258
GPT-o3, 201, 203
GPT-o3-Mini, 205
GPT-o4-Mini, 205–206
GPT-o series, 205
Grok, 11
Grok 3 Mini, 9
Groq, 275, 276
grounded computer use, 213
grounding modalities in primary modality, 160, 165, 272
ground-truth
  for evaluation, 60
  for multimodal systems, 168
  pulling data via, 35, 37
  in text-to-SQL workflow, 34
guardrails
  behavior alignment and, 12
  caching, 19
  for coding agents, 183
  instructional alignment and, 12
GUI automation, 221

H

HAL, 187
hallucination
  experimentation and risk of, 284
  fine-tuning to prevent, 225
  reasoning token retention and, 200
  recall and, 72
  self-reported confidence and, 243
Hamming distance, 43
handoffs, 144–147
hard-negatives, 278
heuristics/rules for evaluation, 60
Hubs, 105–118
Hubspot, 105, 110–111
HuggingFace, 184, 185, 217, 226, 230, 239, 254, 263
human intelligence, 195–196
human-in-the-loop component, 53–54
Humanity's Last Exam (HLE), 205–206, 215
hyperparameters, 256

I

image encoding, 160, 167
image generation, 159, 184–187
in-batch negatives, 278
Inception, 178, 180, 182
in-context learning, 12, 14

indexer
indexing evidence example, 39–41
in RAG systems, 31, 32
inference parameters, 17–19
instruction adherence, 131
instructional alignment, 11, 131, 132, 179–180
instruction training, 246–247
intelligence
increasing system, 195
of reasoning models, 204, 209
seven pillars of, 195–196
interleaved reasoning, 211, 212, 215
interrupt function, 53–55
intersection over union (IoU), 218
intuitive reasoning, 133, 198
inverse document frequency (IDF), 127
iteration by AI engineers, 285

J

Jina, 69, 75
joint modeling, 160, 165–167, 175
JSON outputs, 23, 176, 177, 235

K

keyword-based retrieval, 123
Kimi Audio Instruct, 272
Kimi K2-Instruct, 203
Kimi VL, 199, 201

L

LangChain
agent definition with, 145
built-in messages in, 46
chunking documents, 251
tool definition, 90
LangGraph
building workflows with, 39
checkpointer, 90
coding agent, 182
for computer use, 220–223
context engineering with, 197
experimentation via, 66
interrupt function, 53, 54
MemorySaver feature, 55
multi-agent implementation, 145–147
ReAct agent via, 88–89
setup of, 41
LangSmith, 119–121
language
common workflow language, 39
as pillar of intelligence, 196
large language models (LLMs)
as agent component, 25
agents vs., 90
agent testing by size of, 130
autoencoding/autoregressive, 5–6
calibration of, 231–238
choice for SQL generation, 62–65
comparing reasoning by, 205–206
context engineering inputs for, 198
context windows, 7–10
deterministic nature of, 19
discursive reasoning by, 133
family tree of LLM tasks, 10–12
fine-tuning, 225–226
gates in, 77
grading rubric, 96–99
instruction adherence by, 131, 179–180
overconfident, 132–133, 225
overview of, 3–10
as probabilistic, 4
running prompts across multiple, 77
special features, 17–25
system builds, 3
testing tool use by, 151
text-to-SQL workflow, 32–57
tokens as input for, 4–5
training for chat-based, 246–247
workflows, 25–28
See also tasks, LLM
latency
cache effectiveness and, 21

comparing fine-tuning latencies, 241, 243, 259
comparing STT and TSS latencies, 274–275
in reasoning models, 208–209
step executor, 139
lead generation, 105–111, 117
lead qualification, 112–113, 117, 148
learning
adding knowledge after initial, 252–253
agent vs. workflow, 104
denoising via diffusion, 164
distillation from larger models, 138
documentation/in-context learning, 12
and parameter count, 6
as pillar of intelligence, 195
reinforcement learning, 204
semantic similiarity in, 67–68
See also training
learning_rate, 256
Llama 3-8B, 264
Llama-3-8B-Instruct, 264
Llama 3.2, 7
Llama 4, 152
Llama 4 Maverick, 131
Llama 4 Scout, 78, 81, 82, 97, 151, 276
LLaVA, 166, 167, 168, 175
logic, additions of, 45
logic gray areas, 15
logprobs, 235
look_up_evidence, 90–93
LoRA (low-rank adaptation), 254
lost-in-the-middle problem, 79
low learning rate, 254

M

machine learning, 226
MacPaw dataset, 215, 216, 217
Manhattan distance, 43
masked language modeling, 283
MathQA, 206–209, 215
Matryoshka embeddings, 279–281
max token window chunking, 248
MCP (Model Context Protocol), 106–111, 113, 115, 150
memory
agentic, 26, 100–102
long- vs. short-term, 196
management for context engineering, 197
as pillar of intelligence, 195
short-term memory, 8
VRAM, 262, 263
MemorySaver, 55
Mercury, 178–184
Microsoft Phi.1.5B, 174
mid-execution plan changes, 134
mind, extended, 100
MiniLM, 69, 75
Ministral-3b, 151, 152
miscalibration, 225
Mistral, 63
model calibration, 231–238
models with multiple data modes, 160, 165–167
ModernBERT, 230, 236, 237, 239–243
mode to mode conversions, 160, 164–165
monotonic increase of recall, 73
Moondream, 166, 168, 174–176, 187–188
Moonshot, 199, 201, 203, 272
motivation/values, 196
MRR (mean reciprocal rank), 69–70, 73–75
multi-agent architectures, 140, 141–147
multihop tasks, 72
multimodal AI
any-to-any models, 188–190
capacities of, 159
coding agents, 176–188
components of, 160–168
computer use by, 213–215
illusion of, 160, 168
image retrieval case study, 168–173
modalities embedded in single vector space, 160–164
pretraining of, 160–164
and seven pillars of intelligence, 196
visual Q&A case study, 174–176

multiple choice tasks, 60, 77, 228
multiple negatives ranking loss, 278
multiple vs. single agent use, 104, 116–118

N

natural language queries
converted to structured answers, 95–99
limitations of, 159
in text-to-SQL workflow, 34
vector database for, 39, 43
needle-in-a-haystack problem, 8, 79, 80
network-based architecture, 143–147
nodes
and edges, 38, 50
inputs to/outputs from, 45
Mercury coding agent, 178, 181
network-based architecture, 144, 146
predicting data source choice, 77
ReAct agent, 89
retrieval, 42–45
SQL execution node, 49
SQL generation node, 47
state object as input to, 41
noise refinements, 164, 165
non-open AI, 130
non-reasoning models, 201–204, 215
number tasks, 215–216

O

OLlama, 265
OpenAI
cache system, 21
CLIP, 160–164
constrained decoding by, 179
fine-tuning API, 235
reasoning models, 198, 199, 205
tokenization libraries, 249
top_k, 19
training recipe of, 246
visual Q&A with, 175
OpenRouter, 77, 78, 145, 178
open source embedder testing, 75
optical character recognition (OCR), 165
optimization for production, 261
Opus 4.1, 217, 219, 220
ordering prompts, 12–14
Otto experiment, 196
outputs
node, 45
and persistence of reasoning, 201
rubric for grading, 96
structured, 23–24, 32, 46, 47, 48, 58, 63, 95–99, 179, 201, 215–216, 218, 228
overconfidence, 132–133, 225, 234
overlapping chunking, 249–250

P

parameters
fine-tuning, 225, 226, 239
inference parameters, 17–19
instilling policy documents into, 253
parameter count, 6–7
quantization and value changes in, 263
in VRAM, 262
perception, intelligent, 196
performance
benchmarks and real-life, 75
evaluating agent, 95–100
Mercury's coding, 178
and parameter count, 6
prompt caching and, 19
quantization and loss of, 263, 265, 268
reasoning model, 223
semantic similiarity to improve, 67–68
speed of fine-tuning, 238–242
testing embedding match, 69
workflow vs. agent, 99
planning components, 133–135
policy bot agent, 127–133, 218
policy compliance dataset, 123–127, 129
positional bias, 9–10, 156

positive pairs, 277–279
precision of retrieval, 69–72, 152
preference training, 246–247
pre-training, 246
privacy, data, 242–243
probabilistic engines, 4
production, optimization for, 261
prompt caching, 19–23
prompt chaining, 16–17, 32, 80–83
prompt engineering
  for agent efficiency, 100
  augmenting prompts, 32
  chain of thought, 14
  in context engineering, 197
  and context window size, 7
  few-shot learning, 14–15
  prompt chaining, 16–17
  prompt engineering agents, 130
  prompt ordering, 12–14
  for SQL generation, 46–48
  thinking outside the box for, 82
prompt ordering guidelines, 12–14
prompts
  across multiple LLMs, 77
  defined, 25
  influencing supervisors via, 148
  multiple choice, 228
  randomizing question order in, 152
  SQL queries via, 46–48
  static, 62
  tool calls via, 90
PyAutoGUI, 221, 223
Python, 175, 177, 221, 223
Python MCP implementation, 107–109

Q

qualifier, lead, 112–113, 117, 148
quality assurance, 118
quantization, 254, 262–266, 268
queries
  dataset size and generation of, 36
  embedding similarity for, 43
  evaluating accuracy of, 61
  example of natural language, 34
  first correct result of, 73
  follow-up, 56
  natural language, 34, 39
query-context pairs, 277
Qwen, 175, 189, 269
Qwen3-8B, 138, 212, 249, 257, 258

R

RAG (Retrieval Augmented Generation)
  vs. agent performance, 99
  converting RAG workflow to agent, 88–104
  defined, 31
  evaluating text-to-SQL workflow, 61–77
  for FAQ agent, 124
  with re-ranking addition, 169
  stateful vs. stateless, 57
  in text-to-SQL workflow, 32–56
random noise cleanup, 164, 165
rank_ bm25, 128–129
ranking functions, 127
ReAct (Reasoning + Action) agents, 26–28, 88–89, 91, 124, 127, 135–136, 182, 210–212
“readers,” LLM, 5
reasoning, 195, 198
reasoning models
  benchmarking, 198–209
  chain of thought used by, 14
  evidence to guide reasoning, 34
  fine-tuning Qwen3, 252–257
  how to use in practice, 224
  low/medium/high effort by, 205–206, 208, 217
  MathQA to prompt, 206–209
  vs. non-reasoning models, 201–204
  prompt caching in, 21
  ReAct agents for, 210–212
  and seven pillars of intelligence, 197
  training, 204
  usefulness of, 223

recall, retrieval, 69, 72, 73, 75–76, 152
recurrent neural networks (RNNs), 160
reference-based evaluation, 60
reference-free evaluation, 60
reflection components, 135–136
regex, 199
reinforcement learning, 204, 246–247
relevant information, 8
re-planning component, 134, 136, 141
re-ranking queries, 169–173
Resend, 115, 140
response generation, 5–6
response quality evaluations, 131–133
retrieval
  BM25 for, 128–129
  in context engineering, 197
  via embedding similarity, 43, 58, 61
  evaluating difficulty by domain, 75
  evaluation of, 68–75
  keyword search algorithms, 127
  metrics for, 69
  multimodal image retrieval, 168–173
  in RAG systems, 31, 32
  testing, 68–75
  in text-to-SQL workflow, 42–45
rigidity, workflow, 133, 134, 137
rubric for grading responses, 96–97
rules for evaluation, 60
run_sql_against_database, 90–94

S

scalability, 225
SDR (sales development representative), 105–118, 148
security for coding agents, 187
self-awareness, 196, 202
semantic content comparisons, 77
semantic few-shot learning, 65
Sentence Transformers, 69
separate modality handling, 160, 168
serializing threads, 55
SERP tool, 153
Sesame, 272
short-term memory, 8
SigLIP, 164, 174
single-shot tasks, 215–216, 220
speculative decoding, 269–272
speech-to-text (STT) models, 261, 272–277
SQL queries
  execution of, 48–52
  generation of, 32
  ground-truth queries, 34, 35
  LLM choices for generating, 62–65
  vs. natural language response, 95
  prompts to generate, 46–48
SQL Query Accuracy (SQA), 61, 63–65
stateful agents, 90, 93
stateful workflows, 53, 55, 57, 58, 196
stateless systems, 87
stateless workflows, 53, 57
state object, 41, 45, 52, 53, 57
static prompts, 13, 62
step executor, 135, 136, 137, 139, 140
stratifying splits, 230
Streamlit, 116–118, 140
string formatting, 49
string tasks, 215–216
structured inputs, 114
structured outputs, 23–24, 63, 95–99, 179, 201, 215–216, 218, 228
style alignment, 11
summarizations, AI, 77–83, 138
supervisor architecture, 141, 142, 148–149
synthetic policy compliance dataset, 123–127, 129
system evaluation, 118

T

targets, evaluation, 60
target statement, 8
task-agnostic distillation, 266, 267
tasks, LLM
  accurate agent address to, 95

alignment, 10–12
bucketed by type, 59
categories of, 59–60
chaining prompts, 16
context engineering for, 112
creative solutions to, 82
fine-tuning tasks, 235
mid-execution plan changes, 134
multihop, 72
single-shot computer use, 215–216
spelling words for language-based, 203
temperature, 17, 18, 19, 46
testing
agents, 104, 116–118
evaluation via, 59
need for, 284
reasoning models, 198–210
retrieval, 68–75
Streamlit for ad-hoc, 116–118
structure for, 59
testing agentic tool use, 149–156
with unanswerable questions, 133
testing set (test set), 226, 227, 230, 235
text decoder, 174
text encoding, 160, 162
text-only AI, 159, 160
text-to-speech (TTS) models, 261, 272–277
text-to-SQL workflow, 32–57, 61–77
TF-IDF (term frequency inverse document frequency), 127
thinking models, 14
throughput, 239–241, 268
tiktoken, 249, 250–251
time to first token, 19, 21
tokens
in any-to-any models, 188
as basic LLM input, 4–5
in context window, 7
inference parameters for, 17
logprobs for token confidence, 235
Mercury's production of, 178
positional bias and number of, 9–10
reasoning, 199, 200, 201
time to first token, 19
token probabilities, 243–244
token window chunking, 248
tool calls
agent remembrance of past, 93
by coding agents, 176, 180
evaluating agent calls, 119
for handoffs, 147
information needed for, 155
measuring number of, 94–95
SQL agent statistics on, 94
testing agentic, 153–154
tool call function for agents, 24–25, 89–90, 95
ToolMessages, 182
tools
agent decisions about, 25, 88, 130
ALWAYS USE prompt for, 130
choosing the right, 107
for computer use LLM, 221–222
evaluating agent tool use, 130–133
handoff tools, 144–145
integrated for context engineering, 197
MCP for flexible discovery of, 106
Moondream coding agent tool, 187–188
positional bias in selecting, 156
for SQL generation agent, 89–95
structured output, 23
supervisor (tool-calling) architecture, 141
testing agentic tool use, 149–156
top_k, 17, 19, 114, 235
top_p, 18, 19
traceability, 119–121
training
adding knowledge after initial, 252–253
for distillation, 268
for embedding models, 281–283
to fine-tune generative models, 246–247
performance/reliability via, 12
of reasoning models, 204
See also learning

training set, 226, 227, 230, 235
Transformer architecture
  autoregressive, 162, 164
  bidirectional text encoder in, 164
  encoder, 161
  joint modeling in, 166, 167
  LLMs based in, 6
  positional bias and, 10
  precursors to, 127
  training recipe for, 246
translation tool, 154
trust
  for coding agents, 187
  in model probabilities, 235
  precision as metric for, 71
  at scale, 117
Twilio app, 277

U

unanswerable question testing, 133
underconfidence, 233
understanding tasks, 60
Unsloth, 254, 255
unstructured documents, 124
user FAQs, 124

V

validation set, 226, 227, 230, 235
value alignment, 11
values/motivation, 196
variables
  setting LangSmith, 119–120
  workflow, 41
vector databases, 39, 40, 43, 48, 57, 69, 100
vector space, embedding modalities in one, 160–164
ViLT (Vision-and-Language Transformer), 166, 167, 169, 175
vision encoder, 174
vision–language models, 163
Vision Transformer (ViT), 161
visualization of workflows, 50–52
visual Q&A, 159, 174–176, 187, 213–215
visual search, 159
voice bots, 165, 166, 272–277
VRAM (video random access memory), 262, 263

W

web scraping, 153, 154
web search/web crawling service, 107–111
Weights and Biases, 256
word error rate (WER), 273
workflows
  agentic workflows, 133–141
  benefits of rigid, 133
  common language/framework for, 39
  cost vs. agent cost, 94–95
  data generation, 277
  deterministic predefined, 3, 28
  evaluating RAG text-to-SQL, 61–77
  LangGraph to visualize, 50–52
  LLM, 25–28
  nodes and edges, 38
  planning components, 133–135
  RAG text-to-SQL, 32–57
  reflection components, 135–136
  separate modality handling by, 168
  SQL workflow conversion to agent, 87–104
  stateless vs. stateful, 53–54
  tracing execution of, 119
  workflow vs. agent use, 88, 104, 115
“writers,” LLM, 5

Y

yes/no tasks, 215–216